AI Accelerator on IBM Telum Processor Industrial Product*

Cedric Lichtenau
lichtenau@de.ibm.com
IBM Deutschland R&D GmbH
Boeblingen, BW, Germany

Alper Buyuktosunoglu
alperb@us.ibm.com
IBM Research
Yorktown Heights, NY, USA

Ramon Bertran
rbertra@us.ibm.com
IBM Research
Yorktown Heights, NY, USA

Peter Figuli
Peter.Razvan.Figuli@ibm.com
IBM Deutschland R&D GmbH
Boeblingen, BW, Germany

Christian Jacobi
cjacobi@us.ibm.com
IBM Systems
Poughkeepsie, NY, USA

Nikolaos Papandreou
npo@zurich.ibm.com
IBM Research Europe
Rueschlikon, ZH, Switzerland

Haris Pozidis
hap@zurich.ibm.com
IBM Research Europe
Rueschlikon, ZH, Switzerland

Anthony Saporito
saporit@us.ibm.com
IBM Systems
Poughkeepsie, NY, USA

Andrew Sica
andrewsi@us.ibm.com
IBM Systems
Poughkeepsie, NY, USA

Elpida Tzortzatos
elpida@us.ibm.com
IBM Systems
Poughkeepsie, NY, USA

ABSTRACT

IBM Telum is the next generation processor chip for IBM Z and LinuxONE systems. The Telum design is focused on enterprise class workloads and it achieves over 40% per socket performance growth compared to IBM z15. The IBM Telum is the first server-class chip with a dedicated on-chip AI accelerator that enables clients to gain real time insights from their data as it is getting processed.

Seamlessly infusing AI in all enterprise workloads is highly desirable to get real business insight on every transaction as well as to improve IT operation, security, and data privacy. While it would undeniably provide significant additional value, its application in practice is often accompanied by hurdles from low throughput if run on-platform to security concerns and inconsistent latency if run off-platform. The IBM Telum chip introduces an on-chip AI accelerator that provides consistent low latency and high throughput (over 200 TFLOPS in 32 chip system) inference capacity usable by all threads. The accelerator is memory coherent and directly connected to the fabric like any other general-purpose core to support low latency inference while meeting the system's transaction rate. A scalable architecture providing transparent access to AI accelerator functions via a non-privileged general-purpose core instruction further reduces software orchestration and library complexity as well as provides extensibility to the AI functions. On a global bank customer credit card fraud detection model, the AI accelerator achieves 22× speed up in latency compared to a general purpose core utilizing vector execution units. For the same model, the AI accelerator achieves 116k inferences every second with a latency of only 1.1 msec. As the system is scaled up from one chip to 32 chips, it performs more than 3.5 Million inferences/sec and the latency still stays very low at only 1.2 msec.

This paper briefly introduces the IBM Telum chip and later describes the integrated AI accelerator. IBM Telum's AI accelerator architecture, microarchitecture, integration into the system stack, performance, and power are covered in detail.

CCS CONCEPTS

• **Comp. systems org.** → **Neural networks**; **Systolic arrays**; **Multicore architectures**; **Reliability**; **Availability**; **Processors and memory architectures**; • **Computing methodologies** → **AI**; **Machine learning**; • **Applied computing** → **Enterprise computing**; • **Hardware** → **Power and energy**; **Hardware validation**; **Robustness**.

KEYWORDS

Telum, z16, On-chip AI accelerator, AI on server-class processor, low-latency in-transaction inference, enterprise workload AI

ACM Reference Format:
Cedric Lichtenau, Alper Buyuktosunoglu, Ramon Bertran, Peter Figuli, Christian Jacobi, Nikolaos Papandreou, Haris Pozidis, Anthony Saporito, Andrew Sica, and Elpida Tzortzatos. 2022. AI Accelerator on IBM Telum Processor Industrial Product. In *The 49th Annual International Symposium on Computer Architecture (ISCA '22), June 18–22, 2022, New York, NY, USA.* ACM, New York, NY, USA, 17 pages. https://doi.org/10.1145/3470496.3533042

*This paper is part of the Industry Track of ISCA 2022's program.

ISCA '22, June 18–22, 2022, New York, NY, USA

ACM ISBN 978-1-4503-8610-4/22/06…$15.00
https://doi.org/10.1145/3470496.3533042

1 INTRODUCTION

IBM Z systems are a central part of large enterprise IT infrastructure in industries like banking, retail and logistics [52, 97]. Enterprise workloads are an ever evolving mix of established technologies and new technologies. For example, it is not uncommon for an enterprise workload to be composed of a variety of programs written in COBOL, Java and Python on Node.js. These workloads consist largely of transactional requests as well as batch processing operations. There are several attributes that are traditionally associated with enterprise workloads. First, there is performance at scale. Transaction-based workloads are very sensitive to per thread performance, meaning the ability to finish every single task in a predefined time bound. They are also very sensitive to scalability so that they can scale up to the sheer number of tasks thrown at those systems every second. Enterprise workloads are heterogeneous. For instance, banking workloads are different from logistics workloads, and even within those workloads, there are heterogeneous kinds of programs. Nevertheless, there are some common types of operations that happen across a wide range of applications. Such examples are data sorting [72], compression [2, 109] or cryptography [18]. IBM Z has a long history of implementing hardware accelerators for such tasks in cooperation with the firmware and software teams to enable best possible end to end value from those accelerators. Of course, enterprise workloads are also very sensitive to security. Last but not least, enterprise and mission critical workloads need the best possible reliability and availability.

IBM Telum is the next generation chip for IBM z16 and LinuxONE systems [37] addressing the needs of enterprise class workloads. It provides improved performance, and system capacity in comparison to the predecessor z15 [9, 72, 101]. Figure 1 shows a die photo of the Telum chip. The Telum chip has a die size of 530 mm² and is manufactured in Samsung's 7 nm EUV technology. It contains 22.5B transistors, operates at over 5 GHz and is comprised of 8 SMT2 cores with a new cache hierarchy. Each core has a 128 KB L1 instruction cache, a 128 KB L1 data cache and a 32 MB semi-private L2 cache. The Telum chip has an improved core pipeline and a new cache hierarchy and multi-chip fabric.

IBM z15 [9, 72, 101] and prior system generations [10, 11, 17, 28, 50, 51, 80, 95, 96] implemented shared physical L3 caches on the processor chip, and had a separate chip that implemented a large L4 cache. The Telum design implements all of that logic in a single chip, and opts to quadruple the L2 cache to 32 MB with very low latency. A virtual L3 cache of 256 MB is created by combining the semi-private L2 caches on the chip with a ring infrastructure that supports more than 320 GB/s of bandwidth. Figure 2 shows how a large scale system that connects up to 32 chips is built. Two Telum chips are built on a dual chip module via a synchronous low latency coherency bus tightly connecting 16 cores to a 512 MB virtual cache. Four of those modules get plugged into a four socket drawer which provides eight chips and a virtual L4 cache of 2 GB. Up to four of these drawers can be interconnected into a system with up to 32 chips (200 client-configurable cores) and 8 GB of virtual cache and 40 TB of memory, forming one large scale, coherent, shared memory system.

The Telum chip has an integrated AI accelerator geared towards enabling clients to gain real time insights from their data as it is getting processed. Seamlessly infusing AI in all enterprise workloads is highly desirable where clients use AI on their data to derive insights to improve their businesses. It is also desirable to have AI capability to enable intelligent infrastructure where AI algorithms are used to make the machine more efficient. The Telum chip is the first server class chip that introduces an on-chip AI accelerator providing consistent low latency and high throughput inference capacity usable by all threads to address the requirements of enterprise workloads. Telum's AI accelerator has the capacity to provide real time inference at massive scale so that it can be directly embedded into the transactions at very high bandwidth.

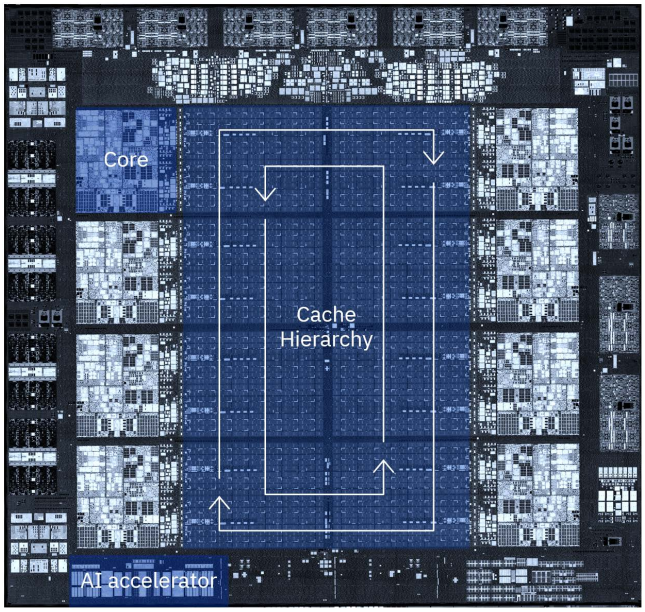

Figure 1: Telum chip die photo highlighting the optimized core, the new cache hierarchy and the AI accelerator.

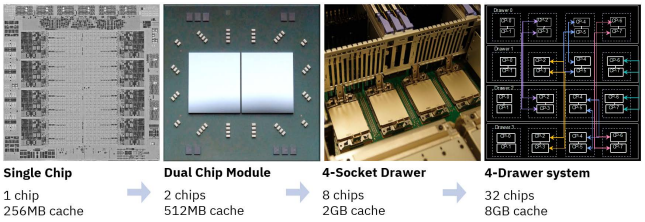

Figure 2: Telum-based system scalability: from a single chip to 32 chips.

Our principal focus in this paper is on the integrated AI accelerator. IBM Telum's AI accelerator architecture, microarchitecture, integration into the system stack, performance, and power are covered in detail. The paper provides research insights on several aspects of the design:

- First server-class chip with a dedicated on-chip AI accelerator and its unique integration challenges.

- First AI accelerator with instruction level abstracted AI functions for multi-generation compatibility and easy upgrade path via firmware updates.
- Enterprise market requirements (security, reliability, latency etc.) manifestation on AI accelerator design and its integration.
- Holistic aspect of the description of the complete design, verification process as well as the integration across the customer stack.
- Focus on key workloads from real banking customers and associated thorough and insightful analysis.
- Use of trusted firmware code that is customized on the fly in hardware for the tensor size.
- Execution of AI operation in interruptible and restartable phases and providing support for transparently transferring an inference process from one machine to another machine/site (disaster failover recovery).
- Dedicated hardware logic for activations, pooling as opposed to just matrix-multiplication.

The rest of the paper is organized as follows. In Section 2, we describe the decisions that shaped the AI accelerator design in Telum chip. In Section 3, we provide integrated AI accelerator design details. We then focus on integration of the AI accelerator into the system stack in Section 4. Section 5 provides results and we conclude in Section 6.

2 INTEGRATED AI ACCELERATOR DESIGN DECISIONS

Increased data volumes for training, advanced algorithms, improvements in computing power and storage enabled significant advancements in deep learning using neural networks [40, 42, 57, 59, 73–76, 81, 84–86, 90, 93, 102, 105–108, 110]. Driven by the advancements in deep learning using neural networks, AI based applications and services have grown tremendously.

Such growth in AI workloads resulted in building specialized systems with hardware accelerators. To meet the computational demands of AI workloads, including commercially available ones [25, 26, 55], a wide range of accelerators from ASIC, FPGA, bit-serial, in-memory designs to large scale systems have been proposed and implemented [5, 7, 19, 20, 22–24, 31, 32, 35, 36, 38, 39, 55, 56, 58, 60–63, 70, 71, 77–79, 82, 83, 87, 91, 100, 103, 104]. Although such a wide-range of design solutions exist, enterprise workloads and AI use cases demand a different solution. Telum's unique solution of an on-chip AI accelerator is driven by several requirements of enterprise world such as low-latency, high-throughput, security, strong memory, virtualization, enterprise class protection mechanisms, availability, reliability, vertical integration between hardware/software, upgrade path, and compatibility across machine generations.

When we look at the enterprise workloads and AI use cases, they roughly fall into two categories. The first category is what we would label business insights where clients can use AI on their business data to derive insights they then use to improve their businesses. Examples include fraud detection on credit cards, customer behavior prediction or supply chain optimization. The second category is labeled as intelligent infrastructure where AI algorithms are used to make the machine more efficient. Examples include intelligent workload placement in an operating system, database query plan optimization or anomaly detection for security.

Let's take credit card processing as an example and specifically credit card fraud detection. Enterprise clients can not reliably achieve the low and consistent latency by sending data from IBM Z to another system (off-platform) for inference. Inference latency around 1-2 msec is a common targeted value by enterprise clients which can not be consistently met by off-platform or even on-platform PCIe attached accelerators. Also, when sending data to an off-platform, it creates all sorts of compatibility, security and regulation concerns. Processed data is often sensitive and personal. Not only the data needs to be in a compatible format but also it has to be encrypted and the security standards need to be audited. This creates additional complexity in an enterprise environment. To overcome such issues, enterprise clients demand the ability to run AI directly embedded into transaction workloads on IBM Z. That way they can score every transaction 100% of the time with the best available model they want to use for that task. Furthermore, off-platform GPU, ASIC or FPGA solutions have other disadvantages such as low-level programming requirements, proprietary drivers, lack of big-endian support, certification costs and limited RAS (Reliability, Availability and Serviceability) support. For example, in the case of RAS, chip target lifetime of GPU and ASIC solutions in the marketplace is far from 100,000 power-on hours that IBM Z processor chip supports [101]. On-platform PCIe attached GPU, ASIC, or FPGA solutions also have similar disadvantages as listed above. In addition, be it on or off-platform, GPU or ASIC solutions entail significant costs for development and qualification of a separate chip.

The other end of the spectrum that implements AI capability directly in the vector execution units of the core has disadvantages as well. When workload switches back and forth between non-AI work and AI work, the AI work can only get the portion of the total capacity that is belonging to that core and latency requirements are not met. In addition, such solutions often are limited in scope to matrix multiplication portion of AI workloads and inherently limit the amount of achievable speed up. Furthermore, in-core solutions expose low-level drivers and requires enterprise clients to update regularly their software stack for additional performance. In view of aforementioned shortcomings of prior solutions, the Telum chip implements a centralized on-chip accelerator directly shared by all the cores. The on-chip AI accelerator approach provides the following attributes:

- By having a centralized accelerator, any core can immediately access the full AI performance without spanning multi-threaded processes with their overhead and variability while the other cores run the customer enterprise workload undisturbed.
- The AI accelerator's compute capacity is optimized to match up the total transaction capacity of the IBM Telum chip. The centralized AI accelerator provides some amount of flexibility on floor planning and where one can place the accelerator on the chip and also how much area one can devote to that accelerator. Between those two considerations, the AI accelerator provides more than 6 TFLOPS of compute

capacity that enables clients to perform AI inference as part of every transaction.

- Enterprise clients use different types of models, ranging from traditional machine learning models like decision trees to various types of neural networks. The accelerator design provides acceleration to the operational types that occur in those different types of AI models.
- The centralized AI accelerator is connected directly to the chip fabric with a data centric design flow. This provides a significantly higher bandwidth compared to the vector unit inside a general purpose core and it does not trash the core local cache hierarchy (e.g. L1) with transient inference data. Furthermore, the accelerator has equally fast access to all L2 on the chip allowing to operate at high data bandwidth on significantly larger models that would not fit inside a core cache.
- The design significantly reduces development and qualification costs of separate chip implementation alternatives be it on or off-platform.
- The design by default supports big-endian and avoids the requirement of big-endian support in closed development drivers or new drivers in other solutions.
- As already mentioned, the security is very important. Just for regulation reasons, keeping data on the platform with a built-in accelerator is a big plus. On-platform solutions also allow to take advantage and inherit the strong memory subsystem, virtualization and memory protection mechanisms and reliability that IBM Z platform implements.
- AI is a quickly evolving field, so the accelerator is designed with extensibility in mind. There is a lot of firmware involved in abstracting how the accelerator works which enables to provide updates, new features and functions with new firmware releases in the future. The firmware also supports fail-over operation between machines of different generations with different versions of the accelerator.
- The on-chip centralized design of the accelerator naturally lends itself to enhancements in future generations of silicon. The design also has the advantage that it is not coupled to the general purpose core design constraints in area, power, and frequency.
- Consolidating the AI operations within the same platform through an integrated AI accelerator results in significant energy savings by avoiding extra IT infrastructure, data movement and provides a major step towards a greener enterprise system.
- The Telum AI accelerator is the first commercial offering using IBM's deep-learning floating-point format (DLFLT) described in [3]. Performance, customer AI models, hardware area/power budget, software stack effort to exploit the hardware are considered in this decision. Our customers use a large set of machine learning models in FP-32/64 and move at their own pace toward deep learning, dictated mostly by regulations and explainability needs. By working across the stack, the design productizes ML in a lower precision (DLFLT-16) while not affecting accuracy for inference and training. DLFLT-16 gives us better accuracy/convergence compared to FP-16 or mixed INT-16/FP-8+FP-32 without retraining at a much higher FLOPS per area.

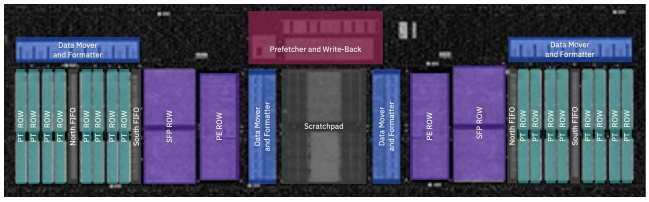

Figure 3: AI accelerator floor plan with the main components highlighted.

3 INTEGRATED AI ACCELERATOR

The integrated AI accelerator spans an area of approximately 7 mm² on the chip (around one third of a general purpose core area) and is synchronously and directly connected to the chip fabric. It is located at chip level (Figure 1), and shares its area with I/O logic requiring a strict physical placement pattern. Although such a choice makes the physical design and closing timing more challenging, it allows easier extensions when new functions are added as there is no fixed bounding box it has to fit into. Figure 3 shows the physical implementation of the accelerator highlighting several components of the design such as processor tiles (PT), processing elements (PE), special function processors (SFP), scratchpad memory, FIFOs, data mover/formatters and prefetcher/write-back. The AI accelerator supports around one million flip-flop equivalent circuits and a 512 KB of scratchpad memory for data prefetching and write-back. The AI accelerator integrates with Telum cores by leveraging a new neural network processing assist instruction in conjunction with firmware running on the core and the accelerator.

3.1 Neural Network Processing Assist (NNPA) Instruction

The Telum design defines a new non-privileged instruction called the Neural Network Processing Assist Instruction. The instruction is a memory to memory CISC instruction, meaning that the operands and the tensor data directly sit in userspace in the program's memory, and registers are used to hold pointers to the required operand descriptors. For instance, for a program that has two source matrices (operands) and a destination matrix in memory, the *NNPA* instruction performs the matrix multiplication of the two source operands and puts the result into the destination operand.

Table 1 lists the supported AI operations and the corresponding *NNPA* functions. Other less often and low compute functions are executed on the general purpose core as they do not contribute significantly to the speed up. The function QAF is used by the software to query the functions supported by the accelerator, supporting seamless migration of AI models between different firmware levels and machine types.

The *elementwise ops* class of operations contains simple tensor elements arithmetic widely used (especially in convolutional neural networks (CNNs)). While these operations can be implemented on most general purpose cores, the accelerator provides significantly higher data bandwidth thus substantially improves performance.

Table 1: AI functions supported by the NNPA Instruction.

Operation Class	Function name
Query op	QAF
Elementwise ops	ADD, SUB, MUL, DIV, MIN, MAX
Activation ops	LOG, EXP, RELU, TANH, SIGMOID
RNN activation ops	LSTMACT, GRUACT
Normalization ops	SOFTMAX, BATCH NORMALIZATION
Pooling ops	AVERAGEPOOL2D, MAXPOOL2D
Systolic ops	FUSED CONVOLUTION,MATMUL-OP, MATMUL-BROADCAST-OP

The *activation ops* class of operations provides an efficient implementation of the most commonly used elementwise compute functions like exponential, ReLU or Sigmoid. The ReLU operation can be parameterized to implement common variants like ReLU9 –clipping the output element to the 9.0 value– without the need to chain multiple AI functions. Avoiding the need to chain multiple AI functions removes the overhead of starting the accelerator and transferring data repetitively to the accelerator. Similarly the *RNN activation ops* class implements all activation functions for LSTM or GRU layers at once, achieving a very large acceleration. The *normalization ops* class implements the SoftMax function –and some other variants of it– as well as 2D-batch normalization on a 4D-tensor for efficiency. The *pooling ops* class supports 2D-pooling operation of type average and maximum.

Finally the *systolic ops* class provides very efficient versions of the 3D-convolution as well as matrix multiplication. These operations are generally responsible for approximately between 65% to 95% of the runtime of inference or training AI models. Enterprise IBM Z clients heavily use machine learning models like decision trees or XGBoost[21] classifiers for explainability reasons. This is supported on the AI accelerator by combining matrix multiplication operations directly with parameterizable elementwise compare operations.

3.2 Microarchitecture

The accelerator is designed to provide enough compute performance to keep up with the maximum sustained system data bandwidth for long running systolic operations while the performance of concurrently running virtual machines or partitions is not noticeably affected. Furthermore, the design matches the peak on-chip data bandwidth for short running elementwise or activation AI functions, and hence gets the maximum possible speed up for these functions. The microarchitecture has two main components: the compute arrays and the data movers. Figure 4 presents a block diagram of the AI accelerator showing the data movers, buffer FIFOs and the prefetch/write back engines surrounding the compute arrays.

3.2.1 Compute Arrays. The integrated AI accelerator delivers more than 6 TFLOPS (for DLFLT-16 FMA operations) per chip which provides over 200 TFLOPS in the 32 chip system[1]. The compute capacity comes from two separate compute arrays. Each compute

[1]The Telum core has 80 GFLOPs for FP32 (smallest FP precision) with two 128-bit pipelines.

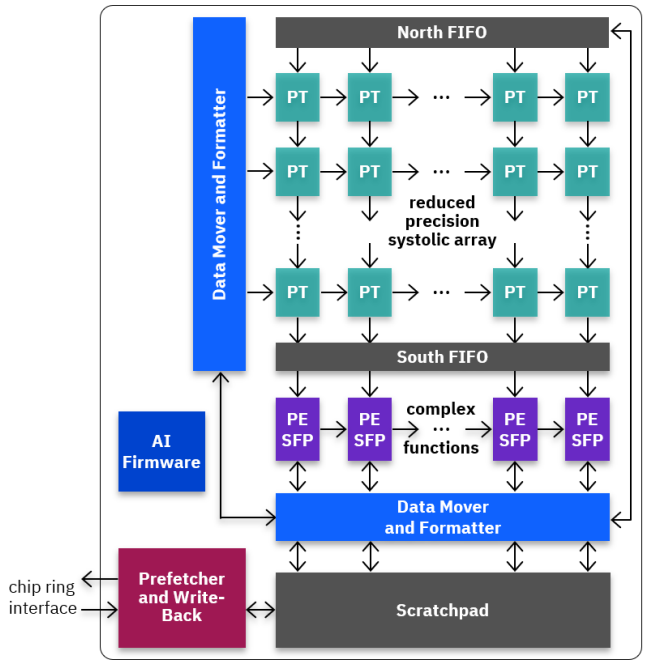

Figure 4: AI accelerator block diagram showing the data movers surrounding the compute arrays composed by the Processor Tiles (PT), the Processing Elements (PE) and the Special Function Processors (SFP).

array is specialized for certain type of AI operations, allowing for higher circuit customization and density as well as lower latency and power since both compute arrays may not be in use. It exploits a systolic dataflow architecture similar to designs in [4, 33, 60, 65, 92]. Discrete synchronization hardware and micro-instructions in the various engines allow for synchronization of the operations within the accelerator and with the general purpose core that initiates the execution of *NNPA* instructions.

The upper array –called the matrix array– consists of 128 Processor Tiles (PTs), which for the sake of simplicity[2] can be regarded as organized as 8 rows of 16 Processor Tiles each. Each row is elementwise connected to the row below. The top row if fed by the second compute array, allows data pre-processing on these inputs, and the bottom row returns results to the second compute array. A second stream of data is provided to the matrix array from the west side and ripples through a processor tile row to support efficient 2D-data computation. This compute array is used, for instance, to implement highly efficient matrix multiplication or convolution operations. Each Processor Tile implements an eight-way SIMD engine optimized for multiply-accumulate operations in DLFLT-16 format[3]. It contains a local register file sized to cover the pipeline depth of the engine and to store a subset of weights for some AI operations.

[2]The actual implementation is a two-corelet design with two individual 8x8 PT arrays for lower latency and increased flexibility.

[3]Deep learning optimized FP format (DLFLT-16), which has better quality and convergence of trained networks in comparison to IEEE FP-16 or bfloat16 formats, is described in [3].

The second array is the data processing and activation array which consists of 32 processor tiles organized as two rows of 16 engines. One row consists of engines named Processing Elements (PE). These PEs are eight-way SIMD engines focused on area and power efficient implementation for arithmetic, logical, look-up and type conversion functions in DLFLT-16 format. The other row consists of 16 engines named Special Function Processors (SFP). The SFP engine is a superset of the PE. Besides supporting the same eight-way SIMD operations as the PE, it can also execute these operations on DLFLT-32 elements with four-way SIMD by keeping the same overall datapath width. The SFP also supports horizontal operations like shifting left/right across engines or computing a sum-across all elements of all SFP engines. This compute array is used either exclusively for all non-systolic functions or for data preparation and gathering for systolic functions.

3.2.2 Data Movers. In order to make the maximum efficient use of the compute arrays and keep latency low for short AI operations, major design effort is invested into the data flow and on the fly data manipulation around these arrays. The AI accelerator data flow starts in the intelligent data prefetch block in red in Figure 4. This block is responsible for just-in-time loading of the data into the 512 KB scratchpad. The scratchpad is organized in multiple sections to enable double-buffering of data and compute streams to allow overlapping of prefetching, compute and write-back phases to maximize parallelism within the accelerator and increase the overall performance. The translated physical addresses for input and output data are provided by the firmware running on the general purpose core. Through the highly predictable access pattern, this *remote* address translation does not hurt latency or throughput and has the significant advantage of removing hardware area and the large coherency complexity of a translation engine in the accelerator. In addition, this design decision also has been a huge enabler to complete the accelerator design on product schedule. The engine fetches the data from the L2 cache through the ring into the accelerator and then results go back the same way, with a read bandwidth of 120+ GB/s.

Data from the scratchpad gets distributed into the input stages of the compute arrays by the data movers. Along that data path, data formatters ensure that the data arrives at the PT compute engines in exactly the format and layout required by the AI operation executed. If needed, additional data manipulation is done by the complex function compute array (PE/SFP) before sending data to the PT compute engines directly via the North FIFO or through more complex data movers on the west side (for instance, for highly efficient matrix multiplication and convolution operations). Overall, the data movers provide more than 600 GB/s of data bandwidth. Finally, the results are collected from the scratchpad by the writeback engine and stored back to the caches or memory via the chip ring with a write bandwidth of 80+ GB/s.

3.3 Firmware

Firmware runs on the general purpose core (where the *NNPA* instruction is executed) and on the AI accelerator[4]. It is delivered with the hardware and it can be updated in the field. The firmware running on the core is in charge of the memory protection and virtualization operations. It translates the program space virtual addresses of the tensors to physical addresses, and it also performs access checking as it performs that translation. As a result, the AI accelerator inherits all the advanced virtualization and protection capabilities of the core. The core also performs prefetching of the tensor data into the L2 cache, in this manner, the data is readily available when the accelerator requests it. The firmware on the core and the accelerator builds a data pipeline that stages the data from DRAM and other system chips to the L2 cache and eventually into the accelerator. The firmware running in the data movers of the AI accelerator distributes the data within the accelerator to reach maximum data bandwidth, latency and power efficiency by matching the compute performance of the compute engines of the accelerator.

A *NNPA* instruction references one register containing the ID number of the AI function to perform as well as another register containing a pointer to a parameter-block located in program's memory. Based on the AI function requested, the core AI firmware interprets the various fields contained in the parameter-block such as the encoding and the size of the tensors, the memory pointers to the tensors, specific sub-functions to be executed and other parameters of the requested function. If all parameters are valid, it computes accelerator control variables based on the tensor size and starts physical address translations for a chunk of the tensor data that is used first. If no data access exception is detected for the tensor chunk, the firmware requests a physical lock to access the AI accelerator, as it is shared by all cores on the chip. The lock request is done last once all data have been prepared to minimize the amount of time any core uses the accelerator. Once granted, the core firmware sends the control variables to the AI accelerator firmware to trigger its start. Firmware on the core then monitors the accelerator execution and sends further address translations for the remaining chunks of the tensor data as required to complete the AI function. Should an interrupt, a context switch or an error be detected, the core firmware interrupts the execution of the AI accelerator at predefined boundaries and saves the current progress in the parameter-block. A subsequent call of the *NNPA* instruction with the same parameter-block seamlessly resumes the execution until final completion of the AI operation.

With generally one inference server program running per chip in customers' configuration, no significant wait time is expected for the lock to access the AI accelerator. Nevertheless, the *NNPA* instruction is configured to wait for the lock for a maximum time of 1 *msec* before returning. If the lock is granted, the *NNPA* instruction execution on the accelerator has been developed to work in phases – each about 4 μs – to allow interruption and load balancing. Significant effort was put in the firmware layer to guarantee balanced usage between all processors executing an *NNPA* instruction. This affects the fairness of the locking mechanism as well as fair periodic swapping between the queued *NNPA* instructions every 16 μs for continuous progress on all requesters. The overhead to swap between two *NNPA* instructions is less than 1 μs, which represents the time to reconfigure the AI accelerator for a different AI primitive or tensor dimensions.

[4]In IBM Z terminology, low-level firmware that runs on core is called millicode [41] and the one on AI accelerator is called picocode.

The firmware on the AI accelerator consists of multiple concatenated machine code for the 10+ specialized engines in the accelerator. This machine code is tailored on-the-fly in the accelerator's hardware with the control variables generated by the core firmware. Special instructions synchronize the execution of the AI function chunk by chunk between the AI accelerator engines and the core firmware.

Overall, there is significant firmware involved in abstracting how the accelerator works. This enables an easy upgrade path by providing enhancements and new functions with firmware releases in the future.

3.4 Reliability and Security

Zero downtime, reliability and security are a key value proposition of IBM Z systems. This applies also to the AI accelerator and has weighed in to the decision to build an on-chip solution instead of off-chip solution. The reliability concept, error detection and correction for the AI accelerator was approached holistically. The goal is that no detected errors leave the accelerator and written back to memory and the accelerator restarts from an intermediate computation checkpoint. Due to this concept, instead of detecting an error immediately, which requires adding checking logic in the critical path, we delayed the error detection by a few cycles and place the circuits in less-critical design area. Additional logic was added to delay data write-back until error checking was completed. Such a design significantly simplifies the ECC logic and does not affect the critical data/control paths. Overall, the error detection and handling logic accounts for 4% of the AI accelerator area. ECC and parity protection is backed up by the sliced execution of AI functions in phases. Sliced execution allows fine rollback capabilities in case an error is detected. If a hard error is detected, the accelerator is guarded off and the AI process is seamlessly transferred to another chip and execution is resumed. In addition, the L2 cache that AI accelerator depends on has multiple level of error correction and sparing mechanism. Data is physically spread in the L2 arrays to prevent an alpha particle from hitting multiple bits covered by the same parity bit. If an error is detected, extra ECC arrays are accessed for correction, eliminating the power cost to read ECC on every read. L2 arrays have been designed for low leakage. Thanks to the low leakage design point, each L2 contains multiple spare arrays. Spare L2 arrays are used by firmware to transparently replace complete L2 arrays that exceeded the threshold for error correction and line delete.

The Telum chip implements a number of innovations in security that the AI accelerator also leverages. It implements encrypted memory and has a performance improved trusted execution environment. The trusted execution environment [15] enables clients to run containerized workloads in a way such that the hardware ensures that the system administrators and the hypervisor administrators cannot get to the data in those containers. Trusted execution also applies to the AI accelerator where firmware guarantees data access isolation. In addition, hardware prevents access to accelerator internal temporary data and stale data in the scratchpad is removed between accelerator invocations.

3.5 Verification

The AI accelerator with its more than 10 different specialized engines working together is complex. The additional layers of firmware running on the accelerator, the general purpose core, the software library and its integration into AI frameworks and transaction processing workflows add further to the complexity. Several design decisions were taken to reduce risk and complexity. For example, the design team did not implement address translation engine in the accelerator. In addition, only well defined and architected basic AI operations were implemented. In typical deep learning models, a group of operations – generally a convolution followed by activations – repeats over and over. Converting this group to a single AI-function would reduce the accelerator data transfers. However, the convolution represents 95% of the group-runtime. The optimization of the remaining 5% has limited performance benefits compared to the additional verification, library and testing effort as well as potential schedule impact.

Most random instructions and micro-architecture verification was done at the specialized engine level. Arithmetic functions like the floating-point multiply-add were verified in parallel by formal verification methods with an in-house tool [67, 88]. Over 99% of all micro-architecture specification ambiguity, new instruction implementation issues and corner case scenario bugs were found by formal verification.

After completing the engine level verification, more than 90% of the verification time for integration testing was spent on high-level and gate-level models containing just the AI accelerator specialized engines and abstracting all external interfaces. In this environment, all variants of the base AI functions were extensively tested by randomizing dimensions, parameters and 4D-tensor data. On the high-level model, speeds around 1 MHz could be achieved. Furthermore, 8% of the testing time was spent in a model containing a general purpose core and the AI accelerator, abstracting the chip ring connection between the core and the accelerator. This environment is used to verify the core firmware in the smallest viable test environment, maximizing the number of system cycles simulated per second. At this level context-switch scenarios and translation page table changes that require the accelerator to cleanly stop, pause and restart were tested. General firmware performance measurement and tuning was also done at this level. Finally, the remaining 2% of the verification time was spent in a chip/multi-chip full model running at first directed accelerator tests via the *NNPA* instruction on the core. Once confidence was built, the verification moved to microbenchmarks and binary compiled multi-layer AI models running under bare metal Linux. This has been extensively used during pre-silicon phases in order to verify the system level performance, tune the AI firmware and software and validate the interaction with other workloads traditionally running in such systems at the same time. Once the Telum chip hardware was available, the full test-suite previously run for multi-chip verification was rapidly deployed and rerun. A couple of bugs related to very large tensor dimensions were discovered in the firmware.

Figure 5 summarizes in which part of the design and which step of the development the bugs were found. Note that bugs include functional bugs –the AI functions not delivering the correct results– and also performance enhancements and physical design

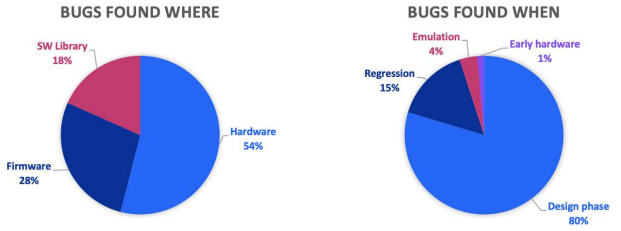

Figure 5: Location and project phase distributions of the bugs found during the AI accelerator design and verification process.

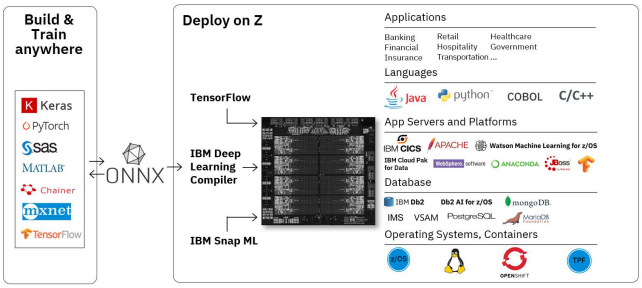

Figure 6: High-level overview of the seamless exploitation of the AI accelerator by the existing enterprise workload stacks and software eco-systems.

changes required for timing. 54% of the bugs found were in the hardware requiring gate level changes in the design and would have hindered correct or efficient functionality of the accelerator. The firmware bugs found (28%) were in the area of large tensor execution chunking. Finally, the software library problems were almost solely performance related. Looking at which phase of the project the bugs were found, the overwhelming majority (80%) were found during the agile sprint phase (2-3 weeks) where the item was designed. 15% of the bugs were found during random test regression, generally within a month of the completed item. The hardware emulation found numerous large tensor (not used in current AI models) bugs. Finally, bugs that escaped to the early hardware were all firmware issues.

4 INTEGRATION OF THE AI ACCELERATOR INTO THE SYSTEM STACK

Vertical integration of the AI solution into the system stack has been a primary focus from concept to implementation. In the baseline, the integration includes a low-level driver and support for a broad open source software eco-system, so that customers can use the tools that their data scientists are already familiar with. Figure 6 shows a high-level overview of that eco-system with familiar logos.

While Telum's AI acceleration focuses mainly on inference, the vertical integration is not limited to it. The integration targets the complete life-cycle management of AI models, from training solution to deployment. AI models can be built in most popular AI frameworks, like TensorFlow [1] or PyTorch [68] and trained anywhere. They can be directly trained on IBM Z systems for compliance and security reasons, but they can also be trained on IBM POWER systems or any other system. It is an IBM design goal to continually instrument popular AI frameworks to utilize the Telum AI accelerator. Initially, deployment on IBM Z and its AI accelerator can be achieved through the following mechanisms:

- Via the ONNX-exporter that converts models built with all major AI framework to the Open Neural-Network Exchange format (ONNX) [8]. ONNX models are then compiled with the Deep Learning Compiler (DLC) to produce an executable for the best performance targeting Telum's AI accelerator. The executable can then be directly integrated into customer application servers and workflows like CICS [47] or Anaconda [6].
- Via IBM Snap ML [30]. Popular machine learning models like decision trees or boosting machines trained using frameworks like scikit-learn [69] or XGBoost [21] can be transformed with the IBM Snap ML library. IBM Snap ML takes advantage of the AI accelerator and provides many model optimizations resulting in significant performance improvements.
- Via TensorFlow framework [1] and its device-model, using the IBM provided AI accelerator device driver. The device driver handles on-the-fly the AI model optimizations, the data alignment and handling for the accelerator and the low-level calls to *NNPA* instructions. It is fully transparent to the user and allows customer to easily enable AI hardware acceleration without any model changes.

The right side of Figure 6 –deploy on Z– shows a typical enterprise stack consisting of the operating systems, container platforms, databases, application servers and applications. As already mentioned in earlier sections, there are use cases for AI at every layer in that stack. The operating system can benefit from AI for intelligent workload placement, databases can optimize their query plans, and then of course, at the higher layer, it targets low-friction integration into client applications to add insights for every transaction and governance of the system. Also, existing applications written in diverse languages such as Java, Python or COBOL using data and AI platforms like Watson Machine Learning [46] or IBM CloudPak for Data [43] seamlessly obtain the AI accelerator performance benefits. Another example of integration is for Db2 [44]. Using Db2 AI for z/OS, Telum accelerated scoring capacity like similarity of rows in tables can be directly accessed via SQL statements and does not require any AI specific knowledge to implement.

4.1 AI Library and Compiler

Although it is possible for an AI framework to use *NNPA* instructions directly, it is much more portable and less error prone to implement a high level programming API. In addition to leveraging the *NNPA* instructions [48], this approach enables the utilization of a common set of software functions that can ease implementation for the AI accelerator in AI frameworks. These functions provide capabilities such as standard approaches to convert tensors from standard formats to the accelerator required format. IBM provides this API via the open source zDNN library [49].

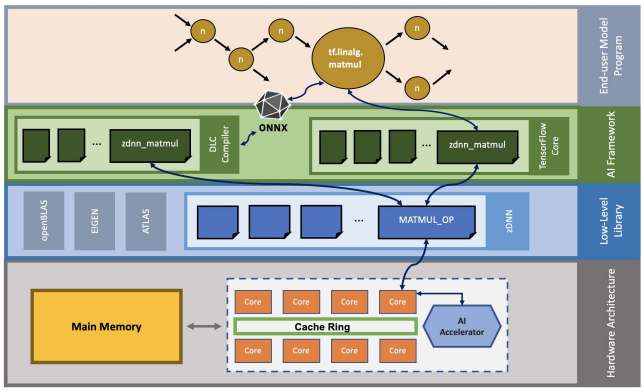

Figure 7: Mapping of a matrix multiply operator from a high level AI model to an operation running on the AI accelerator via the AI framework that leverages the low-level zDNN library that executes the architected *NNPA* instruction.

One approach to utilize the IBM Telum AI accelerator is through the IBM Deep Learning Compiler (DLC). The DLC is an ONNX model compiler which is redesigned recently based on MLIR [66]. The new MLIR-based design allows for a multi-phased optimization which can target and optimize both for general purpose core architecture and the AI accelerator. The flexible design allows for the potential to support multiple levels of AI acceleration as the AI field evolves. In addition to these optimizations, the compiler based approach allows the DLC to precompute and convert weights and other static parameters to the accelerator format at compile time, limiting any run-time conversion to only input features.

Figure 7 depicts how a matrix multiply operator is mapped from an AI model to an operation running on the AI accelerator. The operator is either mapped natively from the AI framework via a device to a corresponding operation in the low-level library (zDNN) and from where the hardware *NNPA* instruction controlling the AI accelerator is called. Alternatively, the AI model is exported to ONNX format and then compiled with DLC, creating an executable linked to the native zDNN library which can be run on the hardware using the AI accelerator.

5 RESULTS

Performance and power evaluation of the AI accelerator is presented in this section. Throughput and power of AI functions are presented with microbenchmarks. End to end application performance results of the AI accelerator using a credit card fraud detection model, a machine learning model and classical deep learning networks are also presented.

5.1 Experimental Setup

The measurements are done on a 32 chips Telum based system running at over 5 GHz. The microbenchmarks are run under Linux Red Hat in guest-1 mode (virtualization level 1). The end to end benchmarks results are produced with the same setup under Linux and under z/OS using IBM Deep Learning Compiler and the zDNN library. For multi-chip measurements, a separate inference server runs on each chip as the cores on a chip can only access its local on-chip AI accelerator. Standard software load-balancing techniques are used to batch inference requests and route the batches to the least busy inference server. Note that the accelerator is designed for inferencing within a transaction with fixed maximum response time (Service Level Agreement). While the accelerator handles the current batch, a new batch is assembled by other cores handling new transactions. This new batch is dispatched to the inference server when the time left to satisfy the maximum response time is approaching the estimated inference time. The IBM Z system architecture provides well predictable task-execution time and hence the delayed inference for batching purpose should never exceed the maximum response time of any transaction.

5.2 Microbenchmark Results

5.2.1 Accelerator Performance. Figure 8 shows the performance speed up as measured on the actual hardware for several AI functions. The x-axis represents the tensors size (number of elements) and the y-axis shows the relative speed up of the AI accelerator throughput in comparison to the functions running on general purpose core utilizing the vector execution units.

For all functions, the AI accelerator performance is a function of the tensor size, and for very small input tensors the speed up is less than one. This is due to the initial start up time of the accelerator as the firmware on the core checks the input parameters to the *NNPA* functions and sets up accelerator control variables and tensor size optimized parameters. Start up time takes between 500 and 2000 core cycles (0.4 usec at 5GHz). Furthermore, small tensors do not fully utilize the hardware parallelism. On the other hand, larger tensors see significant speed up on the AI accelerator compared to the core execution.

For low compute density elementwise functions shown in Figures 8(a)–(h), when the core execution runs out of the L1 cache, it becomes L1 bandwidth dominated. In that scenario, the AI accelerator that is designed for high bandwidth is about an order of magnitude faster compared to the core. This translates into 10× performance advantage for one operand AI functions and 5× for two operands AI functions. For matrix multiplication (Figure 8 (i) and (j)), once the AI accelerator runs in steady state and the startup time becomes negligible, a 56× performance speed up is achievable. Note that the general purpose core uses FP-32 (lowest floating-point precision available) and the accelerator runs in DLFLT-16 mode without affecting the quality of trained networks. Also, when tensor size gets larger, caching and context switch effects due to the longer runtime are observable. Those effects result in non-linearity for the measured experiments and are especially visible in the elementwise functions with little compute to data bandwidth ratio.

5.2.2 Accelerator Power. The addition of the AI accelerator to the chip required the assessment of the power consumption of the accelerator as well as the overall chip power and voltage noise management. Not only the maximum power consumption of the entire chip could be higher if all the resources are fully utilized, but also the expected bursty behavior of the AI accelerator at runtime can cause sudden changes in power consumption, thus stressing the power delivery network.

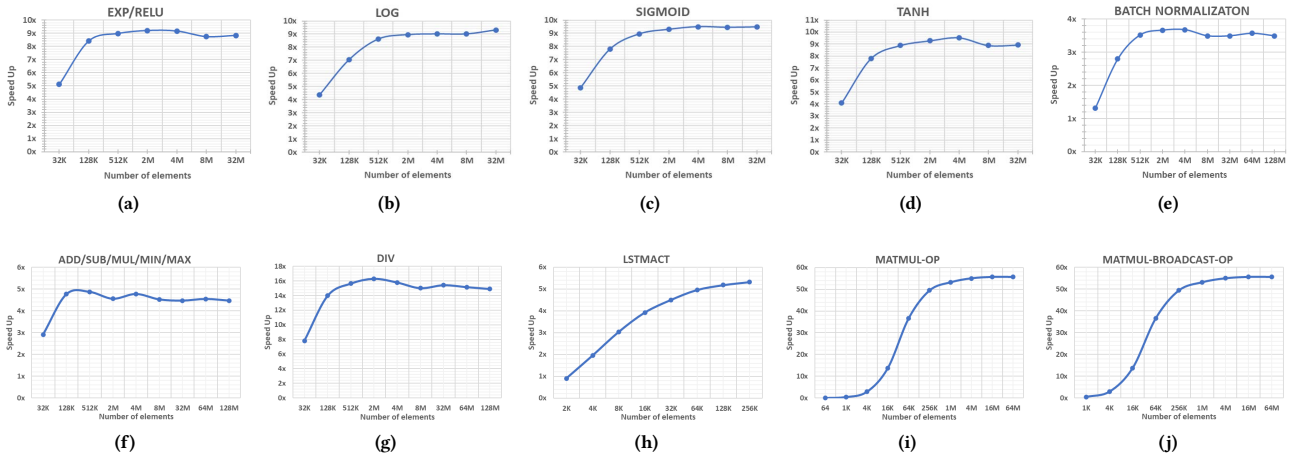

Figure 8: AI accelerator functions throughput speed up in comparison to running those functions on general purpose core vector execution units for different numbers of elements computed.

We accurately characterize the power consumption behavior of the different functions implemented in the AI accelerator using an IBM internal power methodology tightly linked to RTL [29, 53, 54, 89]. By using MicroProbe, a tool which has been used across multiple generations of IBM POWER and Z chips [12–14, 16, 27, 34, 89, 94, 98, 99], specific test cases are generated automatically with the required parameters to perform the VHDL runs in a practical amount of time. 512K-element tensors have been used for most of the NNPA operations, and for operations where tensor dimensions require a specific relationship, the dimensions are adjusted accordingly to ensure valid functional operation. The test cases are run through a detailed VHDL model of a single core plus the AI accelerator and multiple representative cycle windows are collected to evaluate the power consumption.

Figure 9 shows the average power of each function implemented in the AI accelerator normalized to the maximum one. Although not shown in Figure 9, less than 10% of the accelerator power is spent on accessing/writing the data in the L2 and transferring it to/from the AI accelerator. Note that as the accelerator is on-chip, no additional power is required for PHYs or other structures to interface to an off-chip AI accelerator solution in a multi-chip module or on an external card.

Systolic functions such as *FUSED-CONVOLUTION* and *MATMUL* exhibit higher power on average. This is expected since these functions not only load and rearrange data multiple times, but also require significant more compute operations per element. The rest of the operations show similar power on average, except some complex activation functions and the elementwise *DIV* function that exhibit slightly higher average power consumption. Finally, the query function *QAF*, which actually does not perform any computation operation on the AI accelerator shows a significantly lower average power consumption.

Figure 10 presents the power distribution across the different components of the AI accelerator. It shows average power across the entire execution of a single *NNPA* instruction. Specifically, it shows the break down *FUSED-CONVOLUTION* (maximum average power)

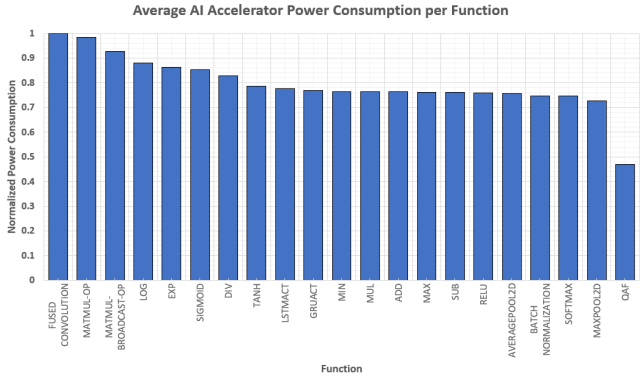

Figure 9: AI accelerator power consumption for each function normalized to *FUSED-CONVOLUTION* function.

and *MAXPOOL-2D* (minimum average power). Data management and movement components are grouped at the left and computation components are grouped at the right of the figure. The green and magenta part of the bars represent the compute power, while the remaining power is consumed by the data movers, the prefetcher and the write-back logic enabling the compute engines. We observe that regardless of the function being executed, the data management components remain fairly similar in power consumption whereas the computation power consumption is very much dependent on the function being performed. Overall, power spent on actual computation is much higher than the power used on managing data, highlighting the efficient design of the AI accelerator.

Fine-grain analysis of power consumption is important to assess the design of the power delivery network and worst case scenario of chip and accelerator power consumption. The bottom chart of Figure 11 shows the fine-grain power behavior of the AI accelerator when executing the *FUSED-CONVOLUTION* function. There are

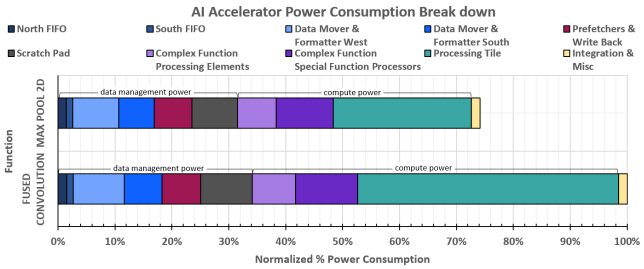

Figure 10: Normalized AI accelerator average power consumption break down for the *FUSED-CONVOLUTION* and the *MAXPOOL-2D* functions.

phases of execution where up to 80% of power is spent on useful computation, while the power consumption of managing data remains quite stable over time. The spikes seen in Figure 11 are mainly due to the different processing steps leading to different activity of the PT array (e.g. data movement, data storage, arithmetic operations) and the phased execution that allows interrupting and resuming an *NNPA* function at well-defined checkpoints. The relationship of power consumption of the AI accelerator versus the core power consumption during the execution of the *FUSED-CONVOLUTION* function is also analyzed and is shown in the top chart of Figure 11. Overall, the results reveal that the efficient and low power design of the accelerator does not constitute a significant portion of the total Telum chip power and therefore, existing robust power management and noise mitigation solutions [99] are deemed to be sufficient with just incremental improvements.

5.3 End to End Benchmark Results

One of the main requirements of the proposed solution is to achieve low latency so that AI can be used in real time and at scale without slowing down transactions. In cooperation with IBM Z enterprise clients, a number of proxy models that reflect real world applications of AI inference are evaluated and results are provided in the following sections.

5.3.1 Credit Card Fraud Detection Model. The credit card fraud detection model uses a recurring neural network that is co-developed with a global bank client. It contains a multi-layer LSTM network, followed by a fully connected layer and normalization. To understand the acceleration opportunities the model has been characterized in detail. Figure 12 presents the runtime break down among the major operators when the model is run on the core. Figure 12 legend also shows the pairing of those operators to AI accelerator functions, which constitute the major part of the overall runtime. Other framework related operators or tensor rearranging functions are also presented which represent a small part of the overall runtime and are run on the core. Most commercial AI acceleration solutions mainly focus only on matrix-multiplication. While this is where most of the time is spent especially on large DL model, for inference and smaller models the relative runtime of activations and other functions like pooling is significant. The dedicated hardware included in the AI accelerator to efficiently handle these functions makes a huge end-to-end performance difference. After model characterization and porting to the accelerator, the batch size effect on throughput and latency are also analyzed.

Batching requests to reduce the number of discrete inferences is a known technique to increase throughput. Figure 13 shows the effect of batching for the AI accelerator for the credit card fraud detection model. Batching has a significant effect on throughput, which is not surprising as small batches do not fully utilize the hardware parallelism in the accelerator and increase the overhead. On the other hand, smaller batches reduce the inference latency. For example, for this particular model, the throughput degradation for batch size of 8 is 48% but the improvement in latency is 60% in comparison to batch size of 128. In general for in-transaction inference, there is a delicate balance between batch size and latency and throughput. The batch size should be selected to meet the required latency while maximizing the possible throughput. We picked a batch-size of 128 to reach an inference latency around 1 msec as commonly targeted value by enterprise clients.

Figure 14 shows latency and throughput results when using different number of chips. When the model is run on a single Telum chip, it can achieve more than 116k inferences every second with a latency of only 1.1 msec. This results in a 22× speed up in latency compared to a general purpose core utilizing vector execution units. Note that if the number of inference requests/sec exceeds the capacity of the AI accelerator, the latency would go asymptotic. Another advantage of a Telum based system is the almost linear scaling of the performance when deploying inference to not just one, but two, eight and up to 32 chips. On a fully configured system with 32 Telum chips, the system can perform more than 3.5 Million inferences/sec at almost the same very low latency of 1.2 msec.

This system scalability is a key characteristic of the IBM Z that is highly valued by customers and played a central point in the design of the AI accelerator. These results show that Telum's AI accelerator has the capacity to provide low latency and real time inference at massive scale, and therefore, it can be used in every transaction at very high bandwidth.

5.3.2 Machine Learning Models. Enterprise IBM Z clients heavily use machine learning (ML) models like random forests or gradient boosting machines, such as XGBoost [21], in addition to deep neural networks. Such ML models comprise multiple decision trees which are scored individually and then combined to produce the inference output.

Scoring a decision tree requires conditional traversal of its nodes until a leaf node is reached. While suitable for implementation in a general-purpose CPU core, the highly irregular access operations in tree traversal are not directly amenable to parallelization in the AI accelerator engine. To circumvent this issue, a new set of algorithms that transform tree traversal into a series of matrix multiplication and elementwise comparison operations are devised and implemented similar to the approach in [64]. These operations can be executed very efficiently in the AI accelerator as we have seen earlier. Optimized machine learning in the AI accelerator is provided by IBM Snap ML [30], a freely-distributed library that boosts the speed and efficiency of machine learning training and inference tasks [45]. IBM Snap ML is a drop-in replacement library for sci-kit learn. It significantly speed-ups training and inference of machine learning models through algorithmic changes. Furthermore the

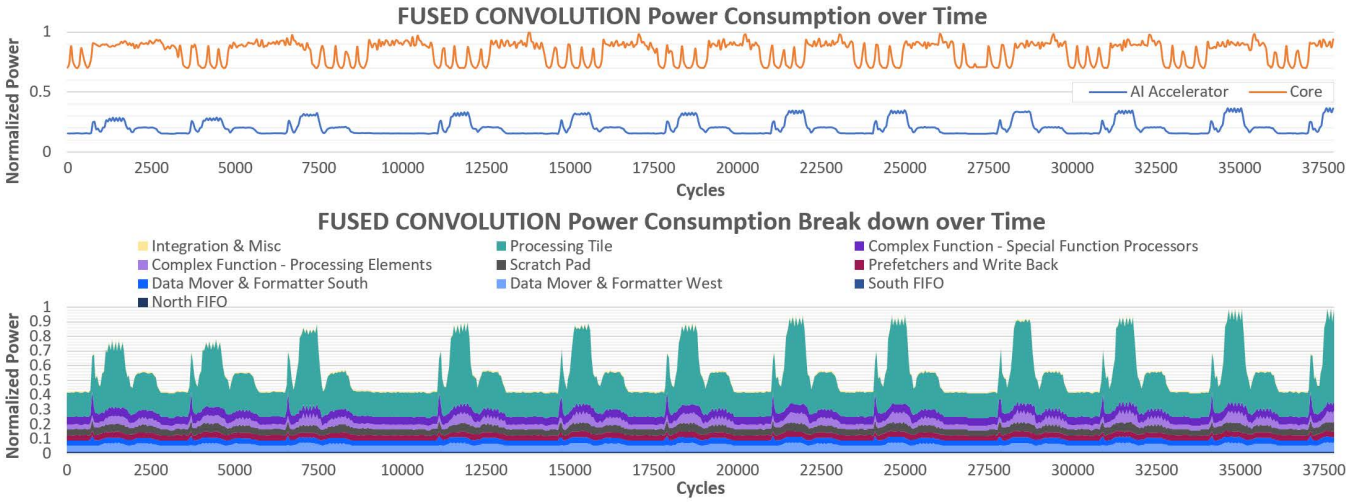

Figure 11: Top: AI accelerator and core power consumption over time for the *FUSED-CONVOLUTION* function. Bottom: AI accelerator power consumption break down over time for the *FUSED-CONVOLUTION* function.

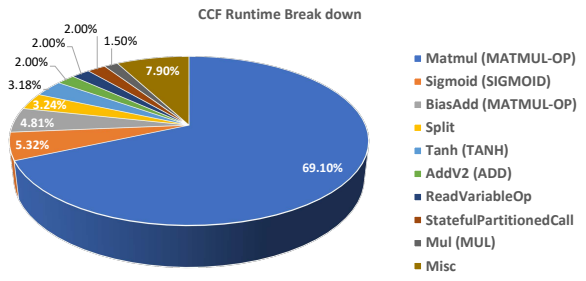

Figure 12: Credit card fraud detection model operator runtime break down when the model is run on the core. The legend shows the pairing of those operators to AI accelerator functions.

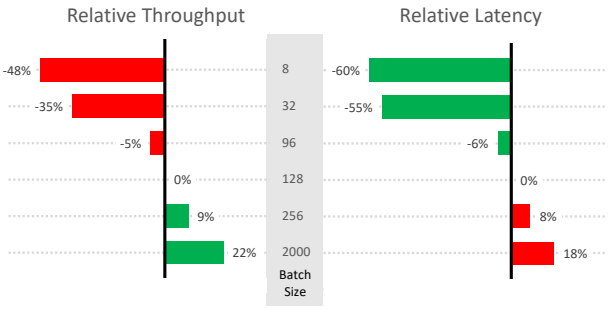

Figure 13: Batching effect on throughput and latency on the credit card fraud detection model.

library has been enhanced to utilize the AI accelerator on the Telum processor.

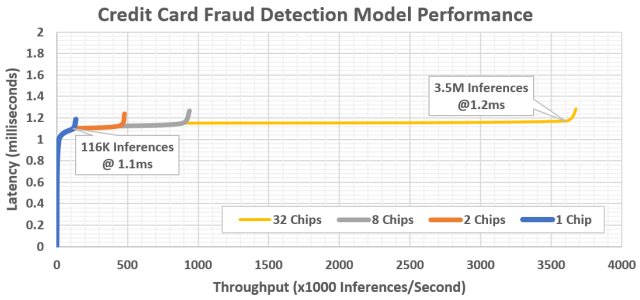

Figure 14: AI accelerator performance on the credit card fraud detection model when using different number of chips, highlighting the system scalability.

Following the *build and train anywhere* philosophy of IBM Z, a user can train their ML model and export it into a familiar format such as *ONNX*, *JSON*, or *PMML*. After that, the model is loaded into Snap ML, transformed into an equivalent tensor-based format as aforementioned, and finally scored in the AI accelerator using the zDNN library API. Figure 15 shows a performance comparison of running inference of the same XGBoost model for a customer transaction prediction use case. Specifically, the task at hand is to predict which customers will make a specific monetary transaction in the future, irrespective of the amount. On one hand, the model is scored in the general purpose cores using the XGBoost inference engine (tree traversal), and on the other hand, it is scored using the AI accelerator via the Snap ML engine (tensor operations). In both cases the same exact model accuracy is achieved. For the particular model, using the AI accelerator results in a sizeable performance improvement that exceeds 4× in latency and 2.5× in throughput.

5.3.3 Deep Learning Networks. Figure 16 summarizes the latency improvement measurements of the AI accelerator in comparison to general purpose core utilizing vector execution units for some

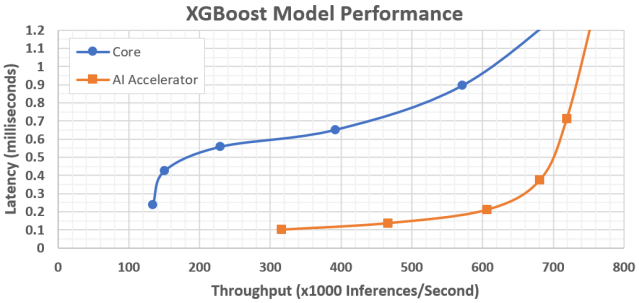

Figure 15: Core and AI accelerator performance on the XGBoost customer transaction prediction model. Model is scored on the general purpose core with XGBoost versus on the AI accelerator with Snap ML.

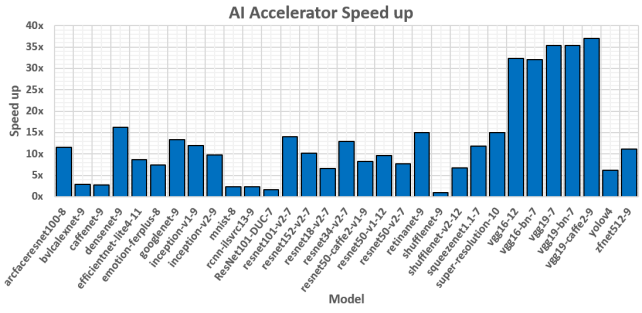

Figure 16: Latency improvement for various AI models when executed on the AI accelerator compared to models running on general purpose core vector execution units.

common deep learning networks. The networks are compiled with the November 2021 version of the IBM Deep Learning Compiler and inference results are compared. Most models see speed up between 5× and 15× and some models see up to 35× speed up. The speed up ratio is heavily dependent on what portion and AI functions of the model can be run on the AI accelerator. The large speed up gain allows enterprise customers to reach low latency inference on 100% of their transactions with enough headroom for a 20% workload growth in the upcoming years. It also allows them to use more complex models resulting in higher accuracy.

6 CONCLUSIONS AND FUTURE WORK

The IBM Telum chip is designed for mission critical enterprise workloads with enhancements in both the traditional aspects of enterprise computing and AI capabilities. A new on-chip AI accelerator is developed and geared towards gaining real time insights from data as it is getting processed.

The centralized on-chip AI accelerator, being connected directly to the chip fabric is accessible by every core. It achieves 6 TFLOPS per chip to harmonize with peak on-chip data bandwidth and infuse AI at total transaction capacity. Two levels of firmware (on core and on accelerator) abstract the accelerator and its interplay with threads, system calls, interruptions, caches and data access isolation. Furthermore, it enables updates in the field and compatibility between machine generations. For software, the accelerator is abstracted by a simple *NNPA* instruction while all the heavy work and orchestration is handled under the hood by the accelerator in conjunction with firmware. The AI accelerator is fully integrated into the system stack, including support for a broad open source eco-system of familiar AI frameworks as well as cross-platform integration through an ONNX exporter and the IBM Deep Learning Compiler. Furthermore, direct deployment and optimization of AI models from TensorFlow and IBM Snap ML onto Telum AI accelerator is enabled and fully transparent to the user.

End-to-end system-level experiments considering the whole stack show up to 56× speed up of AI functions compared to general purpose cores. Popular deep learning networks are accelerated up to 35× end-to-end. A credit card fraud detection model achieves more than 116k inferences/sec per chip with a latency of only 1.1 msec, which is 22× faster than general purpose core. A system with 32 Telum chips achieves more than 3.5M inferences/sec with a latency of just 1.2 msec, enabling real-time in-transaction fraud detection at large scale.

We see several opportunities and challenges as part of future work. Supporting INT-8 in an area-power aware manner and its integration into software stack is worth exploring for its added performance. Adding data manipulation and basic training AI functions and supporting incremental retraining in the software stack is also worth exploring. Low precision arithmetic format for ML algorithms is another important area that requires further research. Finally, lack of enough explainability for deep learning models for numeric (non-image) enterprise workload is a unique research challenge that require investigation.

ACKNOWLEDGMENTS

The authors would like to acknowledge and thank the IBM Z AI team, the IBM Z microprocessor hardware and software team as well as the IBM Research team, whose experience, collaboration, and innovation were instrumental in the development of the AI accelerator solution on the IBM Telum Processor.

REFERENCES

[1] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dandelion Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. 2015. TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems. https://www.tensorflow.org/ Software available from tensorflow.org.

[2] Bülent Abali, Bart Blaner, John J. Reilly, Matthias Klein, Ashutosh Mishra, Craig B. Agricola, Bedri Sendir, Alper Buyuktosunoglu, Christian Jacobi, William J. Starke, Haren Myneni, and Charlie Wang. 2020. Data Compression Accelerator on IBM POWER9 and z15 Processors : Industrial Product. In *47th ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2020, Valencia, Spain, May 30 - June 3, 2020*. IEEE, 1–14. https://doi.org/10.1109/ISCA45697.2020.00012

[3] Ankur Agrawal, Bruce M. Fleischer, Silvia M. Mueller, Xiao Sun, Naigang Wang, Jungwook Choi, and Kailash Gopalakrishnan. 2019. DLFloat: A 16-b Floating Point Format Designed for Deep Learning Training and Inference. In *26th IEEE Symposium on Computer Arithmetic, ARITH 2019, Kyoto, Japan, June 10-12, 2019*, Naofumi Takagi, Sylvie Boldo, and Martin Langhammer (Eds.). IEEE, 92–95. https://doi.org/10.1109/ARITH.2019.00023

[4] Ankur Agrawal, Sae Kyu Lee, Joel Silberman, Matthew M. Ziegler, Mingu Kang, Swagath Venkataramani, Nianzheng Cao, Bruce M. Fleischer, Michael Guillorn, Matt Cohen, Silvia Mueller, Jinwook Oh, Martin Lutz, Jinwook Jung, Siyu Koswatta, Ching Zhou, Vidhi Zalani, James Bonanno, Robert Casatuta, Chia-Yu Chen, Jungwook Choi, Howard Haynie, Alyssa Herbert, Radhika Jain, Monodeep Kar, Kyu-Hyoun Kim, Yulong Li, Zhibin Ren, Scot Rider, Marcel Schaal, Kerstin Schelm, Michael Scheuermann, Xiao Sun, Hung Tran, Naigang Wang, Wei Wang, Xin Zhang, Vinay Shah, Brian W. Curran, Vijayalakshmi Srinivasan, Pong-Fei Lu, Sunil Shukla, Leland Chang, and Kailash Gopalakrishnan. 2021. A 7nm 4-Core AI Chip with 25.6TFLOPS Hybrid FP8 Training, 102.4TOPS INT4 Inference and Workload-Aware Throttling. In *IEEE International Solid-State Circuits Conference, ISSCC 2021, San Francisco, CA, USA, February 13-22, 2021*. IEEE, 144–146. https://doi.org/10.1109/ISSCC42613.2021.9365791

[5] Jorge Albericio, Patrick Judd, Tayler H. Hetherington, Tor M. Aamodt, Natalie D. Enright Jerger, and Andreas Moshovos. 2016. Cnvlutin: Ineffectual-Neuron-Free Deep Neural Network Computing. In *43rd ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2016, Seoul, South Korea, June 18-22, 2016*. IEEE Computer Society, 1–13. https://doi.org/10.1109/ISCA.2016.11

[6] Anaconda Inc. 2012. Anaconda Software Distribution. https://docs.anaconda.com/

[7] Aayush Ankit, Izzat El Hajj, Sai Rahul Chalamalasetti, Geoffrey Ndu, Martin Foltin, R. Stanley Williams, Paolo Faraboschi, Wen-mei W. Hwu, John Paul Strachan, Kaushik Roy, and Dejan S. Milojicic. 2019. PUMA: A Programmable Ultra-efficient Memristor-based Accelerator for Machine Learning Inference. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS 2019, Providence, RI, USA, April 13-17, 2019*, Iris Bahar, Maurice Herlihy, Emmett Witchel, and Alvin R. Lebeck (Eds.). ACM, 715–731. https://doi.org/10.1145/3297858.3304049

[8] Junjie Bai, Fang Lu, and Ke Zhang. 2017. Open Neural Network Exchange (ONNX). https://github.com/onnx/onnx

[9] Christopher J. Berry, Brian Bell, Adam Jatkowski, Jesse Surprise, John Isakson, Ofer Geva, Brian Deskin, Mark Cichanowski, Dina Hamid, Chris Cavitt, Gregory Fredeman, Anthony Saporito, Ashutosh Mishra, Alper Buyuktosunoglu, Tobias Webel, Preetham Lobo, Pradeep Parashurama, Ramon Bertran, Dureseti Chidambarrao, David Wolpert, and Brandon Bruen. 2020. 2.7 IBM z15: A 12-Core 5.2GHz Microprocessor. In *2020 IEEE International Solid- State Circuits Conference, ISSCC 2020, San Francisco, CA, USA, February 16-20, 2020*. IEEE, 54–56. https://doi.org/10.1109/ISSCC19947.2020.9063030

[10] Christopher J. Berry, James D. Warnock, John Badar, Dean G. Bair, Erwin Behnen, Brian Bell, Alper Buyuktosunoglu, Chris Cavitt, Pierce Chuang, Ofer Geva, Dina Hamid, John Isakson, Preetham Lobo, Frank Malgioglio, Guenter Mayer, José Luis Neves, Thomas Strach, Jesse Surprise, Christos Vezyrtzis, Tobias Webel, and David Wolpert. 2018. IBM z14 design methodology enhancements in the 14-nm technology node. *IBM J. Res. Dev.* 62, 2/3 (2018), 9:1–9:12. https://doi.org/10.1147/JRD.2018.2800218

[11] Christopher J. Berry, David Wolpert, Christos Vezyrtzis, Richard F. Rizzolo, Sean M. Carey, Yaniv Maroz, Hunter F. Shi, Dureseti Chidambarrao, Christian Jacobi, Anthony Saporito, Thomas Strach, Alper Buyuktosunoglu, Preetham Lobo, Pierce Chuang, Pawel Owczarczyk, Ramon Bertran, Tobias Webel, and Phillip J. Restle. 2019. IBM z14: Processor Characterization and Power Management for High-Reliability Mainframe Systems. *IEEE J. Solid State Circuits* 54, 1 (2019), 121–132. https://doi.org/10.1109/JSSC.2018.2873582

[12] Ramon Bertran, Alper Buyuktosunoglu, Pradip Bose, Timothy J. Slegel, Gerard Salem, Sean M. Carey, Richard F. Rizzolo, and Thomas Strach. 2014. Voltage Noise in Multi-Core Processors: Empirical Characterization and Optimization Opportunities. In *47th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2014, Cambridge, United Kingdom, December 13-17, 2014*. IEEE Computer Society, 368–380. https://doi.org/10.1109/MICRO.2014.12

[13] Ramon Bertran, Alper Buyuktosunoglu, Meeta Sharma Gupta, Marc González, and Pradip Bose. 2012. Systematic Energy Characterization of CMP/SMT Processor Systems via Automated Micro-Benchmarks. In *45th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2012, Vancouver, BC, Canada, December 1-5, 2012*. IEEE Computer Society, 199–211. https://doi.org/10.1109/MICRO.2012.27

[14] Ramon Bertran, Yutaka Sugawara, Hans M. Jacobson, Alper Buyuktosunoglu, and Pradip Bose. 2013. Application-level power and performance characterization and optimization on IBM Blue Gene/Q systems. *IBM J. Res. Dev.* 57, 1/2 (2013), 4. https://doi.org/10.1147/JRD.2012.2227580

[15] Christian Bornträger, Jonathan D. Bradbury, Reinhard Bündgen, Fadi Busaba, Lisa Cranton Heller, and Viktor Mihajlovski. 2020. Secure your cloud workloads with IBM Secure Execution for Linux on IBM z15 and LinuxONE III. *IBM J. Res. Dev.* 64, 5/6 (2020), 2:1–2:11. https://doi.org/10.1147/JRD.2020.3008109

[16] Calvin Bulla, Lluc Alvarez, Miquel Moretó, Ramon Bertran, Alper Buyuktosunoglu, and Pradip Bose. 2018. ChopStiX: Systematic Extraction of Code-Representative Microbenchmarks. In *2018 IEEE International Symposium on Workload Characterization, IISWC 2018, Raleigh, NC, USA, September 30 - October 2, 2018*. IEEE Computer Society, 80–81. https://doi.org/10.1109/IISWC.2018.8573473

[17] Fadi Busaba, Michael A. Blake, Brian W. Curran, Michael F. Fee, Christian Jacobi, Pak-kin Mak, Brian R. Prasky, and Craig R. Walters. 2012. IBM zEnterprise 196 microprocessor and cache subsystem. *IBM J. Res. Dev.* 56, 1 (2012), 1. https://doi.org/10.1147/JRD.2011.2173962

[18] James A. Busby, Edward N. Cohen, E. Anne Dames, Jessica Doherty, Silvio Dragone, Dave Evans, Michael J. Fisher, Nihad Hadzic, Christoph Hagleitner, Arthur J. Higby, Michael D. Hocker, Luanne S. Jagich, Michael J. Jordan, Richard Kisley, Kirk D. Lamb, Mark D. Marik, Jimmie Mayfield, Thomas E. Morris, Thomas D. Needham, William Santiago-Fernandez, Volker Urban, Tamas Visegrady, and Klaus Werner. 2020. The IBM 4769 Cryptographic Coprocessor. *IBM J. Res. Dev.* 64, 5/6 (2020), 3:1–3:11. https://doi.org/10.1147/JRD.2020.3008145

[19] Srimat T. Chakradhar, Murugan Sankaradass, Venkata Jakkula, and Srihari Cadambi. 2010. A dynamically configurable coprocessor for convolutional neural networks. In *37th International Symposium on Computer Architecture (ISCA 2010), June 19-23, 2010, Saint-Malo, France*, André Seznec, Uri C. Weiser, and Ronny Ronen (Eds.). ACM, 247–257. https://doi.org/10.1145/1815961.1815993

[20] Tianshi Chen, Zidong Du, Ninghui Sun, Jia Wang, Chengyong Wu, Yunji Chen, and Olivier Temam. 2014. DianNao: a small-footprint high-throughput accelerator for ubiquitous machine-learning. In *Architectural Support for Programming Languages and Operating Systems, ASPLOS 2014, Salt Lake City, UT, USA, March 1-5, 2014*, Rajeev Balasubramonian, Al Davis, and Sarita V. Adve (Eds.). ACM, 269–284. https://doi.org/10.1145/2541940.2541967

[21] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, USA, August 13-17, 2016*, Balaji Krishnapuram, Mohak Shah, Alexander J. Smola, Charu C. Aggarwal, Dou Shen, and Rajeev Rastogi (Eds.). ACM, 785–794. https://doi.org/10.1145/2939672.2939785

[22] Yu-Hsin Chen, Joel S. Emer, and Vivienne Sze. 2016. Eyeriss: A Spatial Architecture for Energy-Efficient Dataflow for Convolutional Neural Networks. In *43rd ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2016, Seoul, South Korea, June 18-22, 2016*. IEEE Computer Society, 367–379. https://doi.org/10.1109/ISCA.2016.40

[23] Yunji Chen, Tao Luo, Shaoli Liu, Shijin Zhang, Liqiang He, Jia Wang, Ling Li, Tianshi Chen, Zhiwei Xu, Ninghui Sun, and Olivier Temam. 2014. DaDianNao: A Machine-Learning Supercomputer. In *47th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2014, Cambridge, United Kingdom, December 13-17, 2014*. IEEE Computer Society, 609–622. https://doi.org/10.1109/MICRO.2014.58

[24] Ping Chi, Shuangchen Li, Cong Xu, Tao Zhang, Jishen Zhao, Yongpan Liu, Yu Wang, and Yuan Xie. 2016. PRIME: A Novel Processing-in-Memory Architecture for Neural Network Computation in ReRAM-Based Main Memory. In *43rd ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2016, Seoul, South Korea, June 18-22, 2016*. IEEE Computer Society, 27–39. https://doi.org/10.1109/ISCA.2016.13

[25] Jack Choquette, Wishwesh Gandhi, Olivier Giroux, Nick Stam, and Ronny Krashinsky. 2021. NVIDIA A100 Tensor Core GPU: Performance and Innovation. *IEEE Micro* 41, 2 (2021), 29–35. https://doi.org/10.1109/MM.2021.3061394

[26] Jack Choquette, M.-J. Edward Lee, Ronny Krashinsky, Vishnu Balan, and Brucek Khailany. 2021. 3.2 The A100 Datacenter GPU and Ampere Architecture. In *IEEE International Solid-State Circuits Conference, ISSCC 2021, San Francisco, CA, USA, February 13-22, 2021*. IEEE, 48–50. https://doi.org/10.1109/ISSCC42613.2021.9365803

[27] Pierce I-Jen Chuang, Christos Vezyrtzis, Divya Pathak, Richard F. Rizzolo, Tobias Webel, Thomas Strach, Otto A. Torreiter, Preetham Lobo, Alper Buyuktosunoglu, Ramon Bertran, Michael S. Floyd, Malcolm S. Ware, Gerard Salem, Sean M. Carey, and Phillip Restle. 2017. 26.2 Power supply noise in a 22nm z13™ microprocessor. In *2017 IEEE International Solid-State Circuits Conference, ISSCC 2017, San Francisco, CA, USA, February 5-9, 2017*. IEEE, 438–439. https://doi.org/10.1109/ISSCC.2017.7870449

[28] Brian W. Curran, Christian Jacobi, J. J. Bonanno, D. A. Schroter, K. J. Alexander, A. Puranik, and Markus M. Helms. 2015. The IBM z13 multithreaded microprocessor. *IBM J. Res. Dev.* 59, 4/5 (2015), 1:1–1:13. https://doi.org/10.1147/JRD.2015.2418591

[29] Nagu R. Dhanwada, David J. Hathaway, Victor V. Zyuban, Peng Peng, Karl Moody, William W. Dungan, Arun Joseph, Rahul M. Rao, and Christopher J. Gonzalez. 2013. Efficient PVT independent abstraction of large IP blocks for hierarchical power analysis. In *The IEEE/ACM International Conference on Computer-Aided Design, ICCAD'13, San Jose, CA, USA, November 18-21, 2013*, Jörg Henkel (Ed.). IEEE, 458–465. https://doi.org/10.1109/ICCAD.2013.6691157

[30] Celestine Dünner, Thomas P. Parnell, Dimitrios Sarigiannis, Nikolas Ioannou, Andreea Anghel, Gummadi Ravi, Madhusudanan Kandasamy, and Haralampos Pozidis. 2018. Snap ML: A Hierarchical Framework for Machine Learning. In *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, December 3-8, 2018, Montréal, Canada*, Samy Bengio, Hanna M. Wallach, Hugo Larochelle, Kristen Grauman, Nicolò Cesa-Bianchi, and Roman Garnett (Eds.). Curran Associates, Inc., 250–260. https://proceedings.neurips.cc/paper/2018/hash/

eecca5b6365d9607ee5a9d336962c534-Abstract.html
[31] Schuyler Eldridge, Amos Waterland, Margo I. Seltzer, Jonathan Appavoo, and Ajay Joshi. 2015. Towards General-Purpose Neural Network Computing. In *2015 International Conference on Parallel Architectures and Compilation, PACT 2015, San Francisco, CA, USA, October 18-21, 2015*. IEEE Computer Society, 99–112. https://doi.org/10.1109/PACT.2015.21
[32] Clément Farabet, Berin Martini, B. Corda, Polina Akselrod, Eugenio Culurciello, and Yann LeCun. 2011. NeuFlow: A runtime reconfigurable dataflow processor for vision. In *IEEE Conference on Computer Vision and Pattern Recognition, CVPR Workshops 2011, Colorado Springs, CO, USA, 20-25 June, 2011*. IEEE Computer Society, 109–116. https://doi.org/10.1109/CVPRW.2011.5981829
[33] Bruce M. Fleischer, Sunil Shukla, Matthew M. Ziegler, Joel Silberman, Jinwook Oh, Vijayalakshmi Srinivasan, Jungwook Choi, Silvia Mueller, Ankur Agrawal, Tina Babinsky, Nianzheng Cao, Chia-Yu Chen, Pierce Chuang, Thomas W. Fox, George Gristede, Michael Guillorn, Howard Haynie, Michael Klaiber, Dongsoo Lee, Shih-Hsien Lo, Gary W. Maier, Michael Scheuermann, Swagath Venkataramani, Christos Vezyrtzis, Naigang Wang, Fanchieh Yee, Ching Zhou, Pong-Fei Lu, Brian W. Curran, Leland Chang, and Kailash Gopalakrishnan. 2018. A Scalable Multi- TeraOPS Deep Learning Processor Core for AI Training and Inference. In *2018 IEEE Symposium on VLSI Circuits, Honolulu, HI, USA, June 18-22, 2018*. IEEE, 35–36. https://doi.org/10.1109/VLSIC.2018.8502276
[34] Eric J. Fluhr, Rahul M. Rao, Howard Smith, Alper Buyuktosunoglu, and Ramon Bertran. 2018. IBM POWER9 circuit design and energy optimization for 14-nm technology. *IBM J. Res. Dev.* 62, 4/5 (2018), 4:1–4:11. https://doi.org/10.1147/JRD.2018.2846158
[35] Jeremy Fowers, Kalin Ovtcharov, Michael Papamichael, Todd Massengill, Ming Liu, Daniel Lo, Shlomi Alkalay, Michael Haselman, Logan Adams, Mahdi Ghandi, Stephen Heil, Prerak Patel, Adam Sapek, Gabriel Weisz, Lisa Woods, Sitaram Lanka, Steven K. Reinhardt, Adrian M. Caulfield, Eric S. Chung, and Doug Burger. 2018. A Configurable Cloud-Scale DNN Processor for Real-Time AI. In *45th ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2018, Los Angeles, CA, USA, June 1-6, 2018*, Murali Annavaram, Timothy Mark Pinkston, and Babak Falsafi (Eds.). IEEE Computer Society, 1–14. https://doi.org/10.1109/ISCA.2018.00012
[36] Mingyu Gao, Jing Pu, Xuan Yang, Mark Horowitz, and Christos Kozyrakis. 2017. TETRIS: Scalable and Efficient Neural Network Acceleration with 3D Memory. In *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS 2017, Xi'an, China, April 8-12, 2017*, Yunji Chen, Olivier Temam, and John Carter (Eds.). ACM, 751–764. https://doi.org/10.1145/3037697.3037702
[37] Ofer Geva, Christopher J. Berry, Robert J. Sonnelitter, David Wolpert, Adam Collura, Thomas Strach, Di Phan, Cédric Lichtenau, Alper Buyuktosunoglu, Hubert Harrer, Jeffrey A. Zitz, Chad Marquart, Douglas Malone, Tobias Webel, Adam Jatkowski, John Isakson, Dina Hamid, Mark Cichanowski, Michael Romain, Faisal Hasan, Kevin Williams, Jesse Surprise, Chris Cavitt, and Mark Cohen. 2022. IBM Telum: a 16-Core 5+ GHz DCM. In *IEEE International Solid-State Circuits Conference, ISSCC 2022, San Francisco, CA, USA, February 20-26, 2022*. IEEE, 46–48. https://doi.org/10.1109/ISSCC42614.2022.9731541
[38] Vinayak Gokhale, Jonghoon Jin, Aysegul Dundar, Berin Martini, and Eugenio Culurciello. 2014. A 240 G-ops/s Mobile Coprocessor for Deep Neural Networks. In *IEEE Conference on Computer Vision and Pattern Recognition, CVPR Workshops 2014, Columbus, OH, USA, June 23-28, 2014*. IEEE Computer Society, 696–701. https://doi.org/10.1109/CVPRW.2014.106
[39] Song Han, Xingyu Liu, Huizi Mao, Jing Pu, Ardavan Pedram, Mark A. Horowitz, and William J. Dally. 2016. EIE: Efficient Inference Engine on Compressed Deep Neural Network. In *43rd ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2016, Seoul, South Korea, June 18-22, 2016*. IEEE Computer Society, 243–254. https://doi.org/10.1109/ISCA.2016.30
[40] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep Residual Learning for Image Recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016*. IEEE Computer Society, 770–778. https://doi.org/10.1109/CVPR.2016.90
[41] Lisa Cranton Heller and Mark S. Farrell. 2004. Millicode in an IBM zSeries processor. *IBM J. Res. Dev.* 48, 3-4 (2004), 425–434. https://doi.org/10.1147/rd.483.0425
[42] Gao Huang, Zhuang Liu, Laurens van der Maaten, and Kilian Q. Weinberger. 2017. Densely Connected Convolutional Networks. In *2017 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2017, Honolulu, HI, USA, July 21-26, 2017*. IEEE Computer Society, 2261–2269. https://doi.org/10.1109/CVPR.2017.243
[43] IBM Cloud. 2019. https://www.ibm.com/products/cloud-pak-for-data
[44] IBM Db2. 1993. https://www.ibm.com/products/db2-database
[45] IBM Research. 2020. https://snapml.readthedocs.io
[46] IBM Watson. 2020. https://www.ibm.com/products/machine-learning-for-zos
[47] IBM Z. 1969. https://www.ibm.com/it-infrastructure/z/cics
[48] IBM Z. 2021. https://github.com/IBM/zDNN/blob/main/samples/simple_add.c
[49] IBM Z. 2021. https://github.com/IBM/zDNN
[50] Christian Jacobi, Anthony Saporito, Martin Recktenwald, Aaron Tsai, Ulrich Mayer, Markus M. Helms, Adam Collura, Pak-kin Mak, Robert J. Sonnelitter, Michael A. Blake, Tim Bronson, Arthur O'neill, and Vesselina K. Papazova. 2018. Design of the IBM z14 microprocessor. *IBM J. Res. Dev.* 62, 2/3 (2018), 8:1–8:11. https://doi.org/10.1147/JRD.2018.2798718
[51] Christian Jacobi, Timothy J. Slegel, and Dan F. Greiner. 2012. Transactional Memory Architecture and Implementation for IBM System Z. In *45th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2012, Vancouver, BC, Canada, December 1-5, 2012*. IEEE Computer Society, 25–36. https://doi.org/10.1109/MICRO.2012.12
[52] Christian Jacobi and Charles F. Webb. 2020. History of IBM Z Mainframe Processors. *IEEE Micro* 40, 6 (2020), 50–58. https://doi.org/10.1109/MM.2020.3017107
[53] Hans M. Jacobson, Alper Buyuktosunoglu, Pradip Bose, Emrah Acar, and Richard J. Eickemeyer. 2011. Abstraction and microarchitecture scaling in early-stage power modeling. In *17th International Conference on High-Performance Computer Architecture (HPCA-17 2011), February 12-16 2011, San Antonio, Texas, USA*. IEEE Computer Society, 394–405. https://doi.org/10.1109/HPCA.2011.5749746
[54] Hans M. Jacobson, Arun Joseph, Dharmesh Parikh, Pradip Bose, and Alper Buyuktosunoglu. 2014. Empirically derived abstractions in uncore power modeling for a server-class processor chip. In *International Symposium on Low Power Electronics and Design, ISLPED'14, La Jolla, CA, USA - August 11 - 13, 2014*, Yuan Xie, Tanay Karnik, Muhammad M. Khellah, and Renu Mehra (Eds.). ACM, 147–152. https://doi.org/10.1145/2627369.2627619
[55] Norman P. Jouppi, Doe Hyun Yoon, Matthew Ashcraft, Mark Gottscho, Thomas B. Jablin, George Kurian, James Laudon, Sheng Li, Peter C. Ma, Xiaoyu Ma, Thomas Norrie, Nishant Patil, Sushma Prasad, Cliff Young, Zongwei Zhou, and David A. Patterson. 2021. Ten Lessons From Three Generations Shaped Google's TPUv4i : Industrial Product. In *48th ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2021, Valencia, Spain, June 14-18, 2021*. IEEE, 1–14. https://doi.org/10.1109/ISCA52012.2021.00010
[56] Patrick Judd, Jorge Albericio, Tayler H. Hetherington, Tor M. Aamodt, and Andreas Moshovos. 2016. Stripes: Bit-serial deep neural network computing. In *49th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2016, Taipei, Taiwan, October 15-19, 2016*. IEEE Computer Society, 19:1–19:12. https://doi.org/10.1109/MICRO.2016.7783722
[57] Andrej Karpathy, George Toderici, Sanketh Shetty, Thomas Leung, Rahul Sukthankar, and Li Fei-Fei. 2014. Large-Scale Video Classification with Convolutional Neural Networks. In *2014 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2014, Columbus, OH, USA, June 23-28, 2014*. IEEE Computer Society, 1725–1732. https://doi.org/10.1109/CVPR.2014.223
[58] Duckhwan Kim, Jaeha Kung, Sek M. Chai, Sudhakar Yalamanchili, and Saibal Mukhopadhyay. 2016. Neurocube: A Programmable Digital Neuromorphic Architecture with High-Density 3D Memory. In *43rd ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2016, Seoul, South Korea, June 18-22, 2016*. IEEE Computer Society, 380–392. https://doi.org/10.1109/ISCA.2016.41
[59] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. 2017. ImageNet classification with deep convolutional neural networks. *Commun. ACM* 60, 6 (2017), 84–90. https://doi.org/10.1145/3065386
[60] Sae Kyu Lee, Ankur Agrawal, Joel Silberman, Matthew M. Ziegler, Mingu Kang, Swagath Venkataramani, Nianzheng Cao, Bruce M. Fleischer, Michael Guillorn, Matthew Cohen, Silvia M. Mueller, Jinwook Oh, Martin Lutz, Jinwook Jung, Siyu Koswatta, Ching Zhou, Vidhi Zalani, Monodeep Kar, James Bonanno, Robert Casatuta, Chia-Yu Chen, Jungwook Choi, Howard Haynie, Alyssa Herbert, Radhika Jain, Kyu-Hyoun Kim, Yulong Li, Zhibin Ren, Scot Rider, Marcel Schaal, Kerstin Schelm, Michael Scheuermann, Xiao Sun, Hung Tran, Naigang Wang, Wei Wang, Xin Zhang, Vinay Shah, Brian W. Curran, Vijayalakshmi Srinivasan, Pong-Fei Lu, Sunil Shukla, Kailash Gopalakrishnan, and Leland Chang. 2022. A 7-nm Four-Core Mixed-Precision AI Chip With 26.2-TFLOPS Hybrid-FP8 Training, 104.9-TOPS INT4 Inference, and Workload-Aware Throttling. *IEEE J. Solid State Circuits* 57, 1 (2022), 182–197. https://doi.org/10.1109/JSSC.2021.3120113
[61] Shaoli Liu, Zidong Du, Jinhua Tao, Dong Han, Tao Luo, Yuan Xie, Yunji Chen, and Tianshi Chen. 2016. Cambricon: An Instruction Set Architecture for Neural Networks. In *43rd ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2016, Seoul, South Korea, June 18-22, 2016*. IEEE Computer Society, 393–405. https://doi.org/10.1109/ISCA.2016.42
[62] Wenyan Lu, Guihai Yan, Jiajun Li, Shijun Gong, Yinhe Han, and Xiaowei Li. 2017. FlexFlow: A Flexible Dataflow Accelerator Architecture for Convolutional Neural Networks. In *2017 IEEE International Symposium on High Performance Computer Architecture, HPCA 2017, Austin, TX, USA, February 4-8, 2017*. IEEE Computer Society, 553–564. https://doi.org/10.1109/HPCA.2017.29
[63] Abhinandan Majumdar, Srihari Cadambi, Michela Becchi, Srimat T. Chakradhar, and Hans Peter Graf. 2012. A Massively Parallel, Energy Efficient Programmable Accelerator for Learning and Classification. *ACM Trans. Archit. Code Optim.* 9, 1 (2012), 6:1–6:30. https://doi.org/10.1145/2133382.2133388
[64] Supun Nakandala, Karla Saur, Gyeong-In Yu, Konstantinos Karanasos, Carlo Curino, Markus Weimer, and Matteo Interlandi. 2020. A Tensor Compiler

for Unified Machine Learning Prediction Serving. In *14th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2020, Virtual Event, November 4-6, 2020*. USENIX Association, 899–917. https://www.usenix.org/conference/osdi20/presentation/nakandala

[65] Jinwook Oh, Sae Kyu Lee, Mingu Kang, Matthew M. Ziegler, Joel Silberman, Ankur Agrawal, Swagath Venkataramani, Bruce M. Fleischer, Michael Guillorn, Jungwook Choi, Wei Wang, Silvia Mueller, Shimon Ben-Yehuda, James Bonanno, Nianzheng Cao, Robert Casatuta, Chia-Yu Chen, Matt Cohen, Ophir Erez, Thomas W. Fox, George Gristede, Howard Haynie, Vicktoria Ivanov, Siyu Koswatta, Shih-Hsien Lo, Martin Lutz, Gary W. Maier, Alex Mesh, Yevgeny Nustov, Scot Rider, Marcel Schaal, Michael Scheuermann, Xiao Sun, Naigang Wang, Fanchieh Yee, Ching Zhou, Vinay Shah, Brian W. Curran, Vijayalakshmi Srinivasan, Pong-Fei Lu, Sunil Shukla, Kailash Gopalakrishnan, and Leland Chang. 2020. A 3.0 TFLOPS 0.62V Scalable Processor Core for High Compute Utilization AI Training and Inference. In *IEEE Symposium on VLSI Circuits, VLSI Circuits 2020, Honolulu, HI, USA, June 16-19, 2020*. IEEE, 1–2. https://doi.org/10.1109/VLSICircuits18222.2020.9162917

[66] ONNX-MLIR. 2021. https://onnx.ai/onnx-mlir/

[67] Viresh Paruthi. 2010. Large-scale application of formal verification: From fiction to fact. In *Proceedings of 10th International Conference on Formal Methods in Computer-Aided Design, FMCAD 2010, Lugano, Switzerland, October 20-23*, Roderick Bloem and Natasha Sharygina (Eds.). IEEE, 175–180. https://ieeexplore.ieee.org/document/5770947/

[68] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Z. Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, Hanna M. Wallach, Hugo Larochelle, Alina Beygelzimer, Florence d'Alché-Buc, Emily B. Fox, and Roman Garnett (Eds.). Curran Associates, Inc., 8024–8035. https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html

[69] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.

[70] Mateja Putic, Swagath Venkataramani, Schuyler Eldridge, Alper Buyuktosunoglu, Pradip Bose, and Mircea Stan. 2018. Dyhard-DNN: even more DNN acceleration with dynamic hardware reconfiguration. In *Proceedings of the 55th Annual Design Automation Conference, DAC 2018, San Francisco, CA, USA, June 24-29, 2018*. ACM, 14:1–14:6. https://doi.org/10.1145/3195970.3196033

[71] Brandon Reagen, Paul N. Whatmough, Robert Adolf, Saketh Rama, Hyunkwang Lee, Sae Kyu Lee, José Miguel Hernández-Lobato, Gu-Yeon Wei, and David M. Brooks. 2016. Minerva: Enabling Low-Power, Highly-Accurate Deep Neural Network Accelerators. In *43rd ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2016, Seoul, South Korea, June 18-22, 2016*. IEEE Computer Society, 267–278. https://doi.org/10.1109/ISCA.2016.32

[72] Anthony Saporito, Martin Recktenwald, Christian Jacobi, Gerrit Koch, Deanna Postles Dunn Berger, Robert J. Sonnelitter, Craig R. Walters, Jang-Soo Lee, Cédric Lichtenau, Ulrich Mayer, Eduard Herkel, Stefan Payer, Silvia M. Müller, Vesselina K. Papazova, Ekaterina M. Ambroladze, and Timothy C. Bronson. 2020. Design of the IBM z15 microprocessor. *IBM J. Res. Dev.* 64, 5/6 (2020), 7:1–7:18. https://doi.org/10.1147/JRD.2020.3008119

[73] Ruhi Sarikaya and Alper Buyuktosunoglu. 2010. A Unified Prediction Method for Predicting Program Behavior. *IEEE Trans. Computers* 59, 2 (2010), 272–282. https://doi.org/10.1109/TC.2009.122

[74] Ruhi Sarikaya, Canturk Isci, and Alper Buyuktosunoglu. 2010. Runtime workload behavior prediction using statistical metric modeling with application to dynamic power management. In *Proceedings of the 2010 IEEE International Symposium on Workload Characterization, IISWC 2010, Atlanta, GA, USA, December 2-4, 2010*. IEEE Computer Society, 1–10. https://doi.org/10.1109/IISWC.2010.5650339

[75] Ruhi Sarikaya, Canturk Isci, and Alper Buyuktosunoglu. 2013. Runtime Application Behavior Prediction Using a Statistical Metric Model. *IEEE Trans. Computers* 62, 3 (2013), 575–588. https://doi.org/10.1109/TC.2012.25

[76] Pierre Sermanet, David Eigen, Xiang Zhang, Michaël Mathieu, Rob Fergus, and Yann LeCun. 2014. OverFeat: Integrated Recognition, Localization and Detection using Convolutional Networks. In *2nd International Conference on Learning Representations, ICLR 2014, Banff, AB, Canada, April 14-16, 2014, Conference Track Proceedings*, Yoshua Bengio and Yann LeCun (Eds.). arXiv, 1–16. http://arxiv.org/abs/1312.6229

[77] Ali Shafiee, Anirban Nag, Naveen Muralimanohar, Rajeev Balasubramonian, John Paul Strachan, Miao Hu, R. Stanley Williams, and Vivek Srikumar. 2016. ISAAC: A Convolutional Neural Network Accelerator with In-Situ Analog Arithmetic in Crossbars. In *43rd ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2016, Seoul, South Korea, June 18-22, 2016*. IEEE Computer Society, 14–26. https://doi.org/10.1109/ISCA.2016.12

[78] Yakun Sophia Shao, Jason Clemons, Rangharajan Venkatesan, Brian Zimmer, Matthew Fojtik, Nan Jiang, Ben Keller, Alicia Klinefelter, Nathaniel Ross Pinckney, Priyanka Raina, Stephen G. Tell, Yanqing Zhang, William J. Dally, Joel S. Emer, C. Thomas Gray, Brucek Khailany, and Stephen W. Keckler. 2021. Simba: scaling deep-learning inference with chiplet-based architecture. *Commun. ACM* 64, 6 (2021), 107–116. https://doi.org/10.1145/3460227

[79] Hardik Sharma, Jongse Park, Naveen Suda, Liangzhen Lai, Benson Chau, Vikas Chandra, and Hadi Esmaeilzadeh. 2018. Bit Fusion: Bit-Level Dynamically Composable Architecture for Accelerating Deep Neural Network. In *45th ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2018, Los Angeles, CA, USA, June 1-6, 2018*, Murali Annavaram, Timothy Mark Pinkston, and Babak Falsafi (Eds.). IEEE Computer Society, 764–775. https://doi.org/10.1109/ISCA.2018.00069

[80] C. Kevin Shum, Fadi Busaba, and Christian Jacobi. 2013. IBM zEC12: The Third-Generation High-Frequency Mainframe Microprocessor. *IEEE Micro* 33, 2 (2013), 38–47. https://doi.org/10.1109/MM.2013.9

[81] Karen Simonyan and Andrew Zisserman. 2015. Very Deep Convolutional Networks for Large-Scale Image Recognition. In *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, Yoshua Bengio and Yann LeCun (Eds.). arXiv, 1–14. http://arxiv.org/abs/1409.1556

[82] Linghao Song, Xuehai Qian, Hai Li, and Yiran Chen. 2017. PipeLayer: A Pipelined ReRAM-Based Accelerator for Deep Learning. In *2017 IEEE International Symposium on High Performance Computer Architecture, HPCA 2017, Austin, TX, USA, February 4-8, 2017*. IEEE Computer Society, 541–552. https://doi.org/10.1109/HPCA.2017.55

[83] Naveen Suda, Vikas Chandra, Ganesh Dasika, Abinash Mohanty, Yufei Ma, Sarma B. K. Vrudhula, Jae-sun Seo, and Yu Cao. 2016. Throughput-Optimized OpenCL-based FPGA Accelerator for Large-Scale Convolutional Neural Networks. In *Proceedings of the 2016 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays, Monterey, CA, USA, February 21-23, 2016*, Deming Chen and Jonathan W. Greene (Eds.). ACM, 16–25. https://doi.org/10.1145/2847263.2847276

[84] Christian Szegedy, Sergey Ioffe, Vincent Vanhoucke, and Alexander A. Alemi. 2017. Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning. In *Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence, February 4-9, 2017, San Francisco, California, USA*, Satinder P. Singh and Shaul Markovitch (Eds.). AAAI Press, 4278–4284. http://aaai.org/ocs/index.php/AAAI/AAAI17/paper/view/14806

[85] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott E. Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. 2015. Going deeper with convolutions. In *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2015, Boston, MA, USA, June 7-12, 2015*. IEEE Computer Society, 1–9. https://doi.org/10.1109/CVPR.2015.7298594

[86] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna. 2016. Rethinking the Inception Architecture for Computer Vision. In *2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016*. IEEE Computer Society, 2818–2826. https://doi.org/10.1109/CVPR.2016.308

[87] Masakazu Tanomoto, Shinya Takamaeda-Yamazaki, Jun Yao, and Yasuhiko Nakashima. 2015. A CGRA-Based Approach for Accelerating Convolutional Neural Networks. In *IEEE 9th International Symposium on Embedded Multicore/Manycore Systems-on-Chip, MCSoC 2015, Turin, Italy, September 23-25, 2015*. IEEE Computer Society, 73–80. https://doi.org/10.1109/MCSoC.2015.41

[88] Brian W. Thompto and Bodo Hoppe. 2010. Verification for fault tolerance of the IBM system z microprocessor. In *Proceedings of the 47th Design Automation Conference, DAC 2010, Anaheim, California, USA, July 13-18, 2010*, Sachin S. Sapatnekar (Ed.). ACM, 525–530. https://doi.org/10.1145/1837274.1837405

[89] Brian W. Thompto, Dung Q. Nguyen, José E. Moreira, Ramon Bertran, Hans M. Jacobson, Richard J. Eickemeyer, Rahul M. Rao, Michael Goulet, Marcy Byers, Christopher J. Gonzalez, Karthik Swaminathan, Nagu R. Dhanwada, Silvia M. Müller, Andreas Wagner, Satish Kumar Sadasivam, Robert K. Montoye, William J. Starke, Christian G. Zoellin, Michael S. Floyd, Jeffrey Stuecheli, Nandhini Chandramoorthy, John-David Wellman, Alper Buyuktosunoglu, Matthias Pflanz, Balaram Sinharoy, and Pradip Bose. 2021. Energy Efficiency Boost in the AI-Infused POWER10 Processor. In *48th ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2021, Valencia, Spain, June 14-18, 2021*. IEEE, 29–42. https://doi.org/10.1109/ISCA52012.2021.00012

[90] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett (Eds.). Curran Associates, Inc., 5998–6008. https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html

[91] Swagath Venkataramani, Ashish Ranjan, Subarno Banerjee, Dipankar Das, Sasikanth Avancha, Ashok Jagannathan, Ajaya Durg, Dheemanth Nagaraj, Bharat Kaul, Pradeep Dubey, and Anand Raghunathan. 2017. ScaleDeep: A Scalable Compute Architecture for Learning and Evaluating Deep Networks. In *Proceedings of the 44th Annual International Symposium on Computer Architecture, ISCA 2017, Toronto, ON, Canada, June 24-28, 2017*. ACM, 13–26. https://doi.org/10.1145/3079856.3080244

[92] Swagath Venkataramani, Vijayalakshmi Srinivasan, Wei Wang, Sanchari Sen, Jintao Zhang, Ankur Agrawal, Monodeep Kar, Shubham Jain, Alberto Mannari, Hoang Tran, Yulong Li, Eri Ogawa, Kazuaki Ishizaki, Hiroshi Inoue, Marcel Schaal, Mauricio J. Serrano, Jungwook Choi, Xiao Sun, Naigang Wang, Chia-Yu Chen, Allison Allain, James Bonanno, Nianzheng Cao, Robert Casatuta, Matthew Cohen, Bruce M. Fleischer, Michael Guillorn, Howard Haynie, Jinwook Jung, Mingu Kang, Kyu-Hyoun Kim, Siyu Koswatta, Sae Kyu Lee, Martin Lutz, Silvia Mueller, Jinwook Oh, Ashish Ranjan, Zhibin Ren, Scot Rider, Kerstin Schelm, Michael Scheuermann, Joel Silberman, Jie Yang, Vidhi Zalani, Xin Zhang, Ching Zhou, Matthew M. Ziegler, Vinay Shah, Moriyoshi Ohara, Pong-Fei Lu, Brian W. Curran, Sunil Shukla, Leland Chang, and Kailash Gopalakrishnan. 2021. RaPiD: AI Accelerator for Ultra-low Precision Training and Inference. In *48th ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2021, Valencia, Spain, June 14-18, 2021*. IEEE, 153–166. https://doi.org/10.1109/ISCA52012.2021.00021

[93] Subhashini Venugopalan, Marcus Rohrbach, Jeffrey Donahue, Raymond J. Mooney, Trevor Darrell, and Kate Saenko. 2015. Sequence to Sequence - Video to Text. In *2015 IEEE International Conference on Computer Vision, ICCV 2015, Santiago, Chile, December 7-13, 2015*. IEEE Computer Society, 4534–4542. https://doi.org/10.1109/ICCV.2015.515

[94] Christos Vezyrtzis, Thomas Strach, Pierce I-Jen Chuang, Preetham Lobo, Richard F. Rizzolo, Tobias Webel, Pawel Owczarczyk, Alper Buyuktosunoglu, Ramon Bertran, David T. Hui, Susan M. Eickhoff, Michael S. Floyd, Gerard Salem, Sean M. Carey, Stelios G. Tsapepas, and Phillip J. Restle. 2018. Droop mitigation using critical-path sensors and an on-chip distributed power supply estimation engine in the z14™ enterprise processor. In *2018 IEEE International Solid-State Circuits Conference, ISSCC 2018, San Francisco, CA, USA, February 11-15, 2018*. IEEE, 300–302. https://doi.org/10.1109/ISSCC.2018.8310303

[95] Craig R. Walters, David S. Hutton, Edward W. Chencinski, Christine Axnix, Ralf Winkelmann, Michael F. Fee, Anthony Saporito, and Christian Jacobi. 2018. Performance innovations in the IBM z14 platform. *IBM J. Res. Dev.* 62, 2/3 (2018), 7:1–7:11. https://doi.org/10.1147/JRD.2018.2798340

[96] Craig R. Walters, Pak-kin Mak, D. P. D. Berger, Michael A. Blake, Tim Bronson, K. D. Klapproth, A. J. O'Neill, Robert J. Sonnelitter, and Vesselina K. Papazova. 2015. The IBM z13 processor cache subsystem. *IBM J. Res. Dev.* 59, 4/5 (2015), 5:1–5:11. https://doi.org/10.1147/JRD.2015.2428591

[97] Charles F. Webb. 2021. Microprocessor Advances and the Mainframe Legacy. *IEEE Micro* 41, 6 (2021), 68–70. https://doi.org/10.1109/MM.2021.3115871

[98] Tobias Webel, Preetham M. Lobo, Ramon Bertran, Gerard Salem, Malcolm Allen-Ware, Richard F. Rizzolo, Sean M. Carey, Thomas Strach, Alper Buyuktosunoglu, Charles Lefurgy, Pradip Bose, Ricardo Nigaglioni, Timothy J. Slegel, Michael S. Floyd, and Brian W. Curran. 2015. Robust power management in the IBM z13. *IBM J. Res. Dev.* 59, 4/5 (2015), 16:1–16:12. https://doi.org/10.1147/JRD.2015.2446872

[99] Tobias Webel, Preetham M. Lobo, Thomas Strach, Pradeep Bhadravati Parashurama, Srinivas Purushotham, Ramon Bertran, and Alper Buyuktosunoglu. 2020. Proactive power management in IBM z15. *IBM J. Res. Dev.* 64, 5/6 (2020), 15:1–15:12. https://doi.org/10.1147/JRD.2020.3008143

[100] Ofri Wechsler, Michael Behar, and Bharat Daga. 2019. Spring Hill (NNP-I 1000) Intel's Data Center Inference Chip. In *2019 IEEE Hot Chips 31 Symposium (HCS), Cupertino, CA, USA, August 18-20, 2019*. IEEE, 1–12. https://doi.org/10.1109/HOTCHIPS.2019.8875671

[101] David Wolpert, Christopher J. Berry, Brian Bell, Adam Jatkowski, Jesse Surprise, John Isakson, Ofer Geva, Brian Deskin, Mark Cichanowski, Dina Hamid, Chris Cavitt, Gregory Fredeman, Dinesh Kannambadi, Anthony Saporito, Ashutosh Mishra, Alper Buyuktosunoglu, Tobias Webel, Preetham Lobo, Ramon Bertran, Pradeep Bhadravati Parashurama, Dureseti Chidambarrao, Brandon Bruen, Alan P. Wagstaff, Eric Lukes, Sean M. Carey, Hunter F. Shi, Michael Romain, Paul Logsdon, and Ishita Agarwal. 2021. Cores, Cache, Content, and Characterization: IBM's Second Generation 14-nm Product, z15. *IEEE J. Solid State Circuits* 56, 1 (2021), 98–111. https://doi.org/10.1109/JSSC.2020.3030062

[102] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V. Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, Jeff Klingner, Apurva Shah, Melvin Johnson, Xiaobing Liu, Lukasz Kaiser, Stephan Gouws, Yoshikiyo Kato, Taku Kudo, Hideto Kazawa, Keith Stevens, George Kurian, Nishant Patil, Wei Wang, Cliff Young, Jason Smith, Jason Riesa, Alex Rudnick, Oriol Vinyals, Greg Corrado, Macduff Hughes, and Jeffrey Dean. 2016. Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation. *CoRR* abs/1609.08144 (2016), 1–23. arXiv:1609.08144 http://arxiv.org/abs/1609.08144

[103] Chen Zhang, Peng Li, Guangyu Sun, Yijin Guan, Bingjun Xiao, and Jason Cong. 2015. Optimizing FPGA-based Accelerator Design for Deep Convolutional Neural Networks. In *Proceedings of the 2015 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays, Monterey, CA, USA, February 22-24, 2015*, George A. Constantinides and Deming Chen (Eds.). ACM, 161–170. https://doi.org/10.1145/2684746.2689060

[104] Jintao Zhang, Zhuo Wang, and Naveen Verma. 2017. In-Memory Computation of a Machine-Learning Classifier in a Standard 6T SRAM Array. *IEEE J. Solid State Circuits* 52, 4 (2017), 915–924. https://doi.org/10.1109/JSSC.2016.2642198

[105] Wei Zhang, Xiaodong Cui, Ulrich Finkler, George Saon, Abdullah Kayi, Alper Buyuktosunoglu, Brian Kingsbury, David S. Kung, and Michael Picheny. 2019. A Highly Efficient Distributed Deep Learning System for Automatic Speech Recognition. In *Interspeech 2019, 20th Annual Conference of the International Speech Communication Association, Graz, Austria, 15-19 September 2019*, Gernot Kubin and Zdravko Kacic (Eds.). ISCA, 2628–2632. https://doi.org/10.21437/Interspeech.2019-2700

[106] Wei Zhang, Xiaodong Cui, Abdullah Kayi, Mingrui Liu, Ulrich Finkler, Brian Kingsbury, George Saon, Youssef Mroueh, Alper Buyuktosunoglu, Payel Das, David S. Kung, and Michael Picheny. 2020. Improving Efficiency in Large-Scale Decentralized Distributed Training. In *2020 IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP 2020, Barcelona, Spain, May 4-8, 2020*. IEEE, 3022–3026. https://doi.org/10.1109/ICASSP40776.2020.9054065

[107] Xiangyu Zhang, Xinyu Zhou, Mengxiao Lin, and Jian Sun. 2018. ShuffleNet: An Extremely Efficient Convolutional Neural Network for Mobile Devices. In *2018 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2018, Salt Lake City, UT, USA, June 18-22, 2018*. Computer Vision Foundation / IEEE Computer Society, 6848–6856. https://doi.org/10.1109/CVPR.2018.00716

[108] Matthew M. Ziegler, Ramon Bertran, Alper Buyuktosunoglu, and Pradip Bose. 2017. Machine learning techniques for taming the complexity of modern hardware design. *IBM J. Res. Dev.* 61, 4-5 (2017), 13:1–13:14. https://doi.org/10.1147/JRD.2017.2721699

[109] Christian G. Zoellin, Oliver Draese, Jonathan D. Bradbury, Christian Jacobi, Aditya Puranik, Peter Sutton, and Robert Tokumaru. 2018. New database compression assists in the IBM z14 processor. *IBM J. Res. Dev.* 62, 2/3 (2018), 12:1–12:11. https://doi.org/10.1147/JRD.2018.2802069

[110] Barret Zoph and Quoc V. Le. 2017. Neural Architecture Search with Reinforcement Learning. In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*. OpenReview.net, 1–16. https://openreview.net/forum?id=r1Ue8Hcxg